

Ficha 1

Ferramentas de Mapeamento de Redes e de Análise de Performance da Rede

Trabalho Elaborado por:

Diogo Fernando Águia Costa (8aaaaaaa240696@estg.ipp.pt)

Curso de:

LSIRC - Licenciatura Em Segurança Informática Em Redes De Computadores

Disciplina de:

PPR - Projeção e Planeamento de Redes

Docentes:

João Magalhães (jpm@estg.ipp.pt)

Hélio Sousa (hcs@estg.ipp.pt)

Felgueiras, 6 de março de 2026

Conteúdo

1	Exercício 1: Ferramentas de mapeamento de redes e de análise de performance da rede	10
1.1	Descobrir a rede	10
1.2	Nmap	11
1.2.1	O que é a Ferramenta Nmap?	11
1.2.2	Execução dos Testes	13
1.2.3	Resultados	13
1.3	Angry IP Scanner	16
1.3.1	O que é o Angry IP Scanner?	16
1.3.2	Execução dos Testes	17
1.3.3	Resultados	17
1.4	Conclusão e Comparação das Ferramentas	18
2	Exercício 2: Ferramentas de Análise de Performance da Rede	19
2.1	Exercício 2.1	19
2.1.1	TCP vs UDP	19
2.1.2	Preparação das Máquinas Virtuais	20
2.1.3	Execução dos Testes	22
2.1.4	Testes com Protocolo TCP	23
2.1.5	Observação dos Resultados	24
2.1.6	Testes com Protocolo UDP	24
2.1.7	Observação e Análise	25
2.1.8	Comparação dos Testes e Conclusão Final	25
2.2	Exercício 2.2	26

2.3	Exercício 2.3	27
2.3.1	Preparação das Máquinas na Rede Sem Fios	27
2.3.2	Execução dos Testes de Rede Sem Fios	29
2.3.3	Testes com Protocolo TCP	29
2.3.4	Testes com Protocolo UDP	30
2.3.5	Observação dos Resultados Wi-Fi	31
2.3.6	Conclusão Parcial: Análise Exclusiva da Rede Wi-Fi	32
2.3.7	Comparação de Resultados: Wi-Fi vs. Rede Interna Virtual (Ex. 2.1)	32
2.4	Exercício 2.4	33
2.4.1	Metodologia	33
2.4.2	Topologia de Teste (Rede Interna Virtual)	34
2.4.3	Testes com Protocolo TCP	34
2.4.4	Testes com Protocolo UDP	35
2.4.5	Observação e Análise dos Resultados Transacionais	36
2.4.6	Topologia de Teste (Rede via Wi-Fi)	37
2.4.7	Topologia de Teste (Rede via Wi-Fi)	38
2.4.8	Testes com Protocolo TCP	38
2.4.9	Testes com Protocolo UDP	39
2.4.10	Observação de Resultados (Rede Wi-Fi)	40
2.4.11	Conclusão Final(Virtual vs. Wi-Fi)	41
3	Exercício 3: Análise de Desempenho com Iperf	42
3.1	Exercício 3.1	42
3.1.1	Parâmetros e Metodologia	42
3.1.2	Topologia de Teste (Rede Interna Virtual)	43
3.1.3	Testes com Protocolo TCP	43
3.1.4	Conclusão dos Testes TCP (Rede Interna)	47
3.1.5	Testes com Protocolo UDP	48
3.1.6	Conclusão dos Testes UDP (Rede Interna)	52
3.1.7	Conclusão de TCP vs UDP	53
3.1.8	Rede Wi-Fi	53
3.2	Exercício 3.2	53

3.2.1	O que é o campo ToS?	53
3.2.2	Valores de ToS e Parâmetros Iperf	54
3.2.3	Preparação da Infraestrutura	54
3.2.4	Configuração do Router	55
3.2.5	Configuração do Cliente e Servidor	55
3.2.6	Teste de Conectividade	55
3.2.7	Testes	56
3.2.8	Observação e Análise de Resultados (QoS)	57
3.3	Exercício 3.3	58
3.3.1	Preparação das Máquinas	58
3.3.2	Testes	59
3.3.3	Observação dos Resultados	61
3.4	Exercício 3.4	61
3.4.1	Introdução	62
3.4.2	Preparação e Início dos Testes	62
3.4.3	Observação e Análise dos Resultados	66
3.5	Exercício 3.5	67
3.5.1	Testes Realizados	68
3.5.2	Observação dos Resultados	69
3.5.3	Conclusão	70

Lista de Figuras

1.1	Output do comando ipconfig com a identificação da rede	11
1.2	Logótipo da aplicação Nmap	11
1.3	Interface principal do Zenmap (GUI do Nmap)	12
1.4	Configuração do alvo e seleção do perfil de varrimento no Zenmap	12
1.5	Execução do perfil Intense scan na interface do Zenmap	13
1.6	Listagem de <i>hosts</i> detetados na barra lateral do Zenmap	14
1.7	Separador <i>Ports/Hosts</i> com detalhe das portas, estados e serviços	14
1.8	Separador <i>Host Details</i> com informações vitais do dispositivo	15
1.9	Representação gráfica da topologia da rede no Zenmap	15
1.10	Logótipo da aplicação Angry IP Scanner	16
1.11	Menu de seleção de métricas (Fetchers) no Angry IP Scanner	16
1.12	Resultados do varrimento apresentados na interface do Angry IP Scanner	17
2.1	Atribuição do endereço IP 192.168.1.10 à máquina servidor	21
2.2	Atribuição do endereço IP 192.168.1.20 à máquina cliente	21
2.3	Instalação do pacote netperf na máquina servidor	21
2.4	Instalação do pacote netperf na máquina cliente	21
2.5	Teste de conectividade (ping) realizado com sucesso entre as instâncias	22
2.6	Resultado do teste TCP_STREAM com pacotes de 1024 bytes	23
2.7	Resultado do teste TCP_STREAM com pacotes de 2048 bytes	23
2.8	Resultado do teste TCP_STREAM com pacotes de 4096 bytes	23
2.9	Resultado do teste UDP_STREAM com pacotes de 1024 bytes	24
2.10	Resultado do teste UDP_STREAM com pacotes de 2048 bytes	25
2.11	Resultado do teste UDP_STREAM com pacotes de 4096 bytes	25

2.12	Verificação do endereço IP atribuído à máquina(Servidor)	28
2.13	Verificação do endereço IP atribuído à máquina(Cliente)	28
2.14	Validação da conectividade bidirecional (ping) entre as instâncias via rede Wi-Fi	29
2.15	Resultado do teste TCP via Wi-Fi com pacotes de 1024 bytes	29
2.16	Resultado do teste TCP via Wi-Fi com pacotes de 2048 bytes	30
2.17	Resultado do teste TCP via Wi-Fi com pacotes de 4096 bytes	30
2.18	Resultado do teste UDP via Wi-Fi com pacotes de 1024 bytes	30
2.19	Resultado do teste UDP via Wi-Fi com pacotes de 2048 bytes	31
2.20	Resultado do teste UDP via Wi-Fi com pacotes de 4096 bytes	31
2.21	Resultado do teste transacional TCP_RR (1 byte pedido / 1 byte resposta)	34
2.22	Resultado do teste transacional TCP_RR (1 byte pedido / 1024 bytes resposta)	35
2.23	Resultado do teste transacional TCP_RR (1024 bytes pedido / 1024 bytes resposta)	35
2.24	Resultado do teste transacional UDP_RR (1 byte pedido / 1 byte resposta)	35
2.25	Resultado do teste transacional UDP_RR (1 byte pedido / 1024 bytes resposta)	36
2.26	Resultado do teste transacional UDP_RR (1024 bytes pedido / 1024 bytes resposta)	36
2.27	Resultado TCP_RR via Wi-Fi (1 byte pedido / 1 byte resposta)	38
2.28	Resultado TCP_RR via Wi-Fi (1 byte pedido / 1024 bytes resposta)	39
2.29	Resultado TCP_RR via Wi-Fi (1024 bytes pedido / 1024 bytes resposta)	39
2.30	Resultado UDP_RR via Wi-Fi (1 byte pedido / 1 byte resposta)	39
2.31	Resultado UDP_RR via Wi-Fi (1 byte pedido / 1024 bytes resposta)	40
2.32	Resultado UDP_RR via Wi-Fi (1024 bytes pedido / 1024 bytes resposta)	40
3.1	Teste Iperf TCP com duração de 30 segundos	43
3.2	Teste Iperf TCP com duração de 60 segundos	43
3.3	Teste Iperf TCP com 1 conexão nativa	44
3.4	Teste Iperf TCP com 4 conexões paralelas	44
3.5	Teste Iperf TCP com relatório a cada 30 segundos	46
3.6	Teste Iperf TCP com relatório a cada 5 segundos	46
3.7	Teste Iperf UDP com duração de 30 segundos (sem limite de banda definido)	48
3.8	Teste Iperf UDP com duração de 60 segundos (sem limite de banda definido)	48
3.9	Teste Iperf UDP com 1 conexão nativa (sem limite de banda definido)	49
3.10	Teste Iperf UDP com 4 conexões paralelas (sem limite de banda definido)	50

3.11	Teste Iperf UDP com relatório a cada 30 segundos	51
3.12	Teste Iperf UDP com relatório a cada 5 segundos	51
3.13	Topologia de rede com Router intermediário para testes de QoS	54
3.14	Atribuição de IP à interface eth0 do Router	55
3.15	Ativação da interface de rede eth0	55
3.16	Ativação do IP Forwarding para permitir o encaminhamento de pacotes	55
3.17	Configuração do Default Gateway no Cliente/Servidor	55
3.18	Teste de conectividade (Ping) validando a rota através do Router	56
3.19	Teste Iperf sem manipulação do campo ToS	56
3.20	Teste Iperf com ToS configurado para minimizar custo (0x02)	56
3.21	Teste Iperf com ToS configurado para maximizar fiabilidade (0x04)	57
3.22	Teste Iperf com ToS configurado para maximizar débito (0x08)	57
3.23	Teste Iperf com ToS configurado para minimizar atraso (0x10)	57
3.24	Teste Iperf UDP sem manipulação do campo ToS	59
3.25	Teste Iperf UDP com ToS configurado para minimizar custo (0x02)	59
3.26	Teste Iperf UDP com ToS configurado para maximizar fiabilidade (0x04)	60
3.27	Teste Iperf UDP com ToS configurado para maximizar débito (0x08)	60
3.28	Teste Iperf UDP com ToS configurado para minimizar atraso (0x10)	60
3.29	Servidor Iperf a escutar tráfego UDP com Bind (-B) ao IP 192.168.1.10	62
3.30	Teste UDP padrão (Sem parâmetro -b)	62
3.31	Teste UDP forçado a 5 Mbits/sec (-b 5M)	63
3.32	Teste UDP forçado a 10 Mbits/sec (-b 10M)	63
3.33	Teste UDP forçado a 50 Mbits/sec (-b 50M)	63
3.34	Teste UDP forçado a 100 Mbits/sec (-b 100M)	64
3.35	Teste UDP forçado a 200 Mbits/sec (-b 200M)	64
3.36	Teste UDP padrão com relatórios intermédios a cada 5 segundos	65
3.37	Teste UDP a 10 Mbits/sec com relatórios intermédios	65
3.38	Teste UDP a 50 Mbits/sec evidenciando quebras durante os relatórios intermédios	66
3.39	Teste de transferência de ficheiro (Pasta zipada) com a opção -F	68
3.40	Teste TCP com janela definida para 10 KByte	68
3.41	Teste TCP com janela definida para 128 KByte	68
3.42	Teste TCP com janela definida para 512 KByte	69
3.43	Teste TCP com janela definida para 1000 KByte (limitada pelo SO)	69

Lista de Tabelas

2.1	Resultados do <i>throughput</i> TCP em função do tamanho do pacote	24
2.2	Resultados do <i>throughput</i> UDP em função do tamanho do pacote	25
2.3	Resultados do <i>throughput</i> TCP via Wi-Fi	31
2.4	Resultados do <i>throughput</i> UDP via Wi-Fi	32
2.5	Resultados transacionais TCP_RR em função do tamanho do pedido/resposta	36
2.6	Resultados transacionais UDP_RR em função do tamanho do pedido/resposta	37
2.7	Resultados transacionais TCP_RR via Wi-Fi	40
2.8	Resultados transacionais UDP_RR via Wi-Fi	40
3.1	Resultados do iperf (TCP) com variação do tempo de execução	44
3.2	Resultados detalhados para 4 conexões em simultâneo (-P 4)	45
3.3	Resultados para 1 conexão única (-P 1)	45
3.4	Resultados detalhados com intervalo de relatório de 5 segundos (-i 5)	47
3.5	Resultados com intervalo de relatório de 30 segundos (-i 30)	47
3.6	Resultados do iperf (UDP) na variação do tempo de execução	49
3.7	Resultados detalhados para 4 conexões UDP em simultâneo (-P 4)	50
3.8	Resultados para 1 conexão UDP única (-P 1)	50
3.9	Relatório detalhado UDP com intervalo de 5 segundos (-i 5)	52
3.10	Relatório global UDP com intervalo de 30 segundos (-i 30)	52
3.11	Comparação resumida entre os protocolos TCP e UDP na rede interna	53
3.12	Valores do campo ToS e respetivas políticas de encaminhamento	54
3.13	Resultados dos testes TCP com manipulação do campo Type of Service (ToS)	58
3.14	Resultados dos testes UDP com manipulação da prioridade (ToS)	61
3.15	Resultados do teste de stress UDP com injeção de largura de banda crescente	66

3.16 Resultados da transferência de ficheiros reais	69
3.17 Impacto do tamanho da Janela TCP no desempenho da rede	69

Exercício 1: Ferramentas de mapeamento de redes e de análise de performance da rede

O presente exercício tem como foco a pesquisa e utilização de ferramentas, de uso livre, destinadas ao mapeamento de redes. O objetivo principal consiste em executar as ferramentas selecionadas para compreender e avaliar as suas aptidões, nomeadamente no que diz respeito ao desenho do mapa e topologia da rede, bem como à sua capacidade de análise de tráfego.

Para a realização desta demonstração, foram selecionadas duas ferramentas de destaque: o **Nmap** e o **Angry IP Scanner**. Em ambas as aplicações, serão testadas e comparadas as funcionalidades de descoberta de dispositivos (*hosts*), varrimento e mapeamento da infraestrutura da rede local.

Como passo inicial, proceder-se-á à identificação do endereçamento IP e respetiva máscara de sub-rede da máquina de testes, de forma a descobrir o intervalo de rede correto. Posteriormente, será efetuado o varrimento a essa mesma rede, procedendo-se à documentação e análise dos resultados gerados por cada uma das aplicações.

1.1 Descobrir a rede

Numa fase inicial, é necessário abrir o terminal da máquina para descobrir a que rede a mesma se encontra conectada. Para esse efeito, utilizou-se o comando **ipconfig** no sistema operativo Windows, procedendo-se à análise do respetivo *output*.

```
Linha de comandos
Microsoft Windows [Version 10.0.26200.7922]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\diogo>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet:

    Connection-specific DNS Suffix  . : home
    Link-local IPv6 Address . . . . . : fe80::570:878b:f677:ef88%8
    IPv4 Address. . . . . : 192.168.1.236
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.1.1

Ethernet adapter Ethernet 2:

    Connection-specific DNS Suffix  . :
    Link-local IPv6 Address . . . . . : fe80::33:1d4c:2ab3:5490%6
    IPv4 Address. . . . . : 192.168.56.1
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :
```

Figura 1.1: Output do comando ipconfig com a identificação da rede

Através da análise da Figura 1.1, verifica-se que o adaptador principal (*Ethernet adapter Ethernet*) possui o endereço IPv4 **192.168.1.236** e a máscara de sub-rede **255.255.255.0**. Com base nesta informação, determina-se que a rede alvo para o processo de varrimento e mapeamento é a **192.168.1.0/24**.

1.2 Nmap



Figura 1.2: Logótipo da aplicação Nmap

1.2.1 O que é a Ferramenta Nmap?

A ferramenta Nmap pode ser executada tanto via interface de linha de comandos como através da sua interface gráfica de utilizador, oficialmente designada por **Zenmap**. Para a realização dos presentes testes, optou-se pela utilização do Zenmap, uma vez que oferece uma interação mais intuitiva e facilita significativamente a análise e visualização topológica dos resultados.

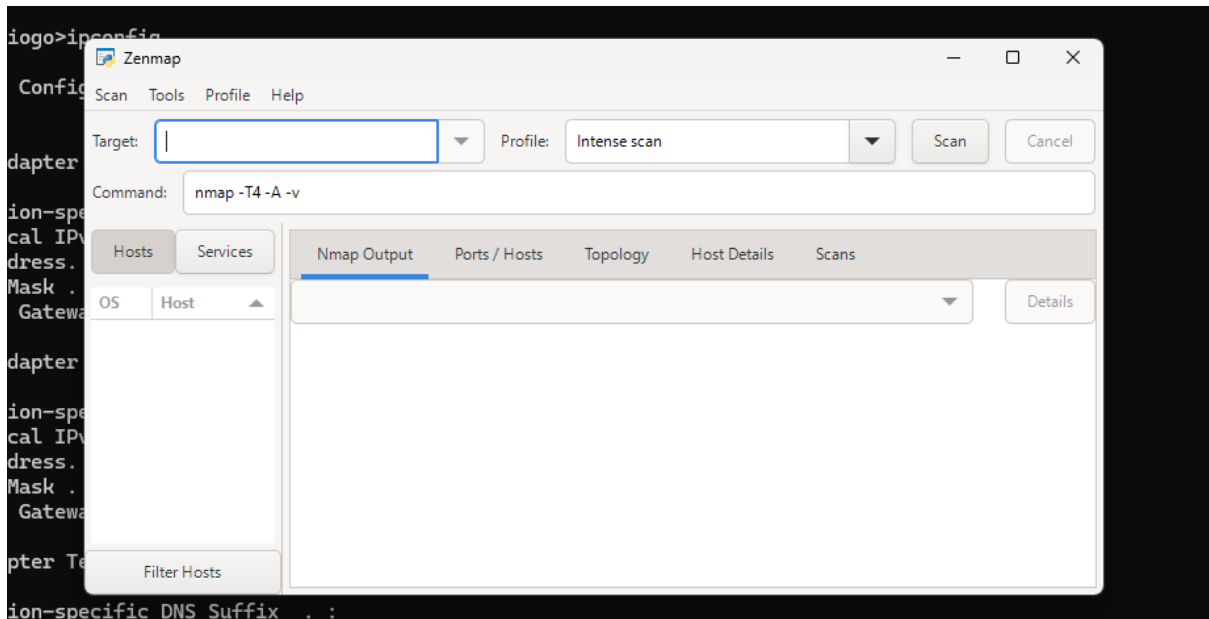


Figura 1.3: Interface principal do Zenmap (GUI do Nmap)

Tendo previamente identificado a rede alvo (**192.168.1.0/24**), o processo de varrimento inicia-se através da inserção deste valor no campo *Target*. De seguida, no campo *Profile*, selecciona-se o perfil de varrimento pretendido. O Nmap disponibiliza diversos perfis predefinidos, cada um configurado com parâmetros distintos para diferentes níveis de profundidade e objetivos de análise.

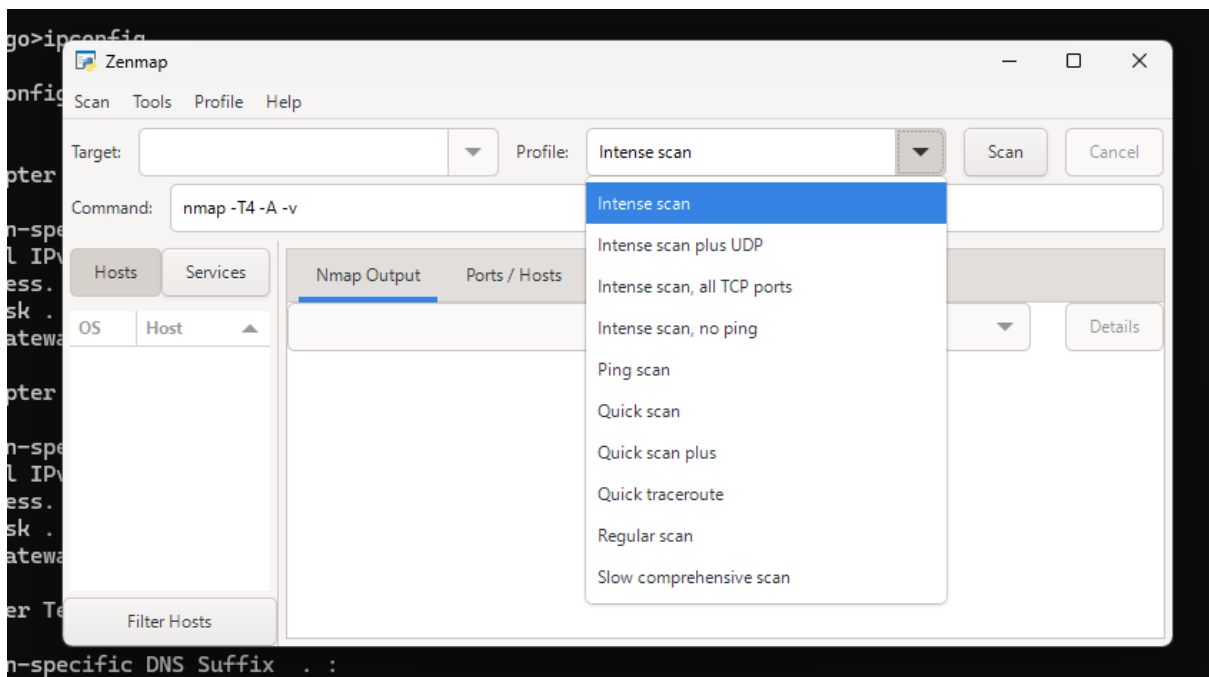


Figura 1.4: Configuração do alvo e seleção do perfil de varrimento no Zenmap

1.2.2 Execução dos Testes

Para a realização dos presentes testes, selecionou-se o perfil *Intense scan*. Este modo executa um varrimento exaustivo e detalhado, englobando não só a descoberta de *hosts*, mas também a deteção do sistema operativo, identificação das versões dos serviços em execução nas portas abertas, execução de *scripts* avançados e *traceroute*.

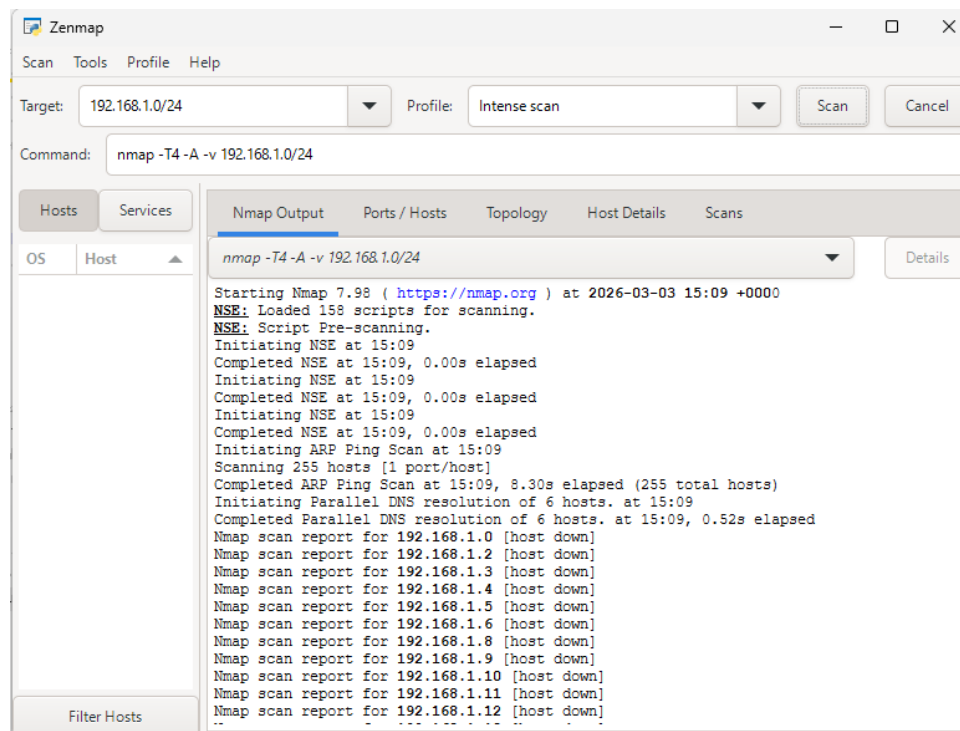


Figura 1.5: Execução do perfil Intense scan na interface do Zenmap

Após a inserção dos parâmetros necessários, iniciou-se o processo de varrimento. Dada a profundidade da análise efetuada pelo *Intense scan*, aguardou-se a conclusão da recolha e processamento de pacotes para posterior análise dos resultados.

1.2.3 Resultados

Após a conclusão do varrimento, foi possível recolher um conjunto diversificado de informações detalhadas sobre a rede. A interface gráfica do Zenmap organiza estes dados de forma estruturada, permitindo a análise das seguintes componentes:

- **Hosts Ativos:** Todos os dispositivos (*hosts*) identificados na rede são listados na barra lateral esquerda da aplicação. Através da seleção individual de cada dispositivo, o utilizador consegue aceder a informações específicas do mesmo.

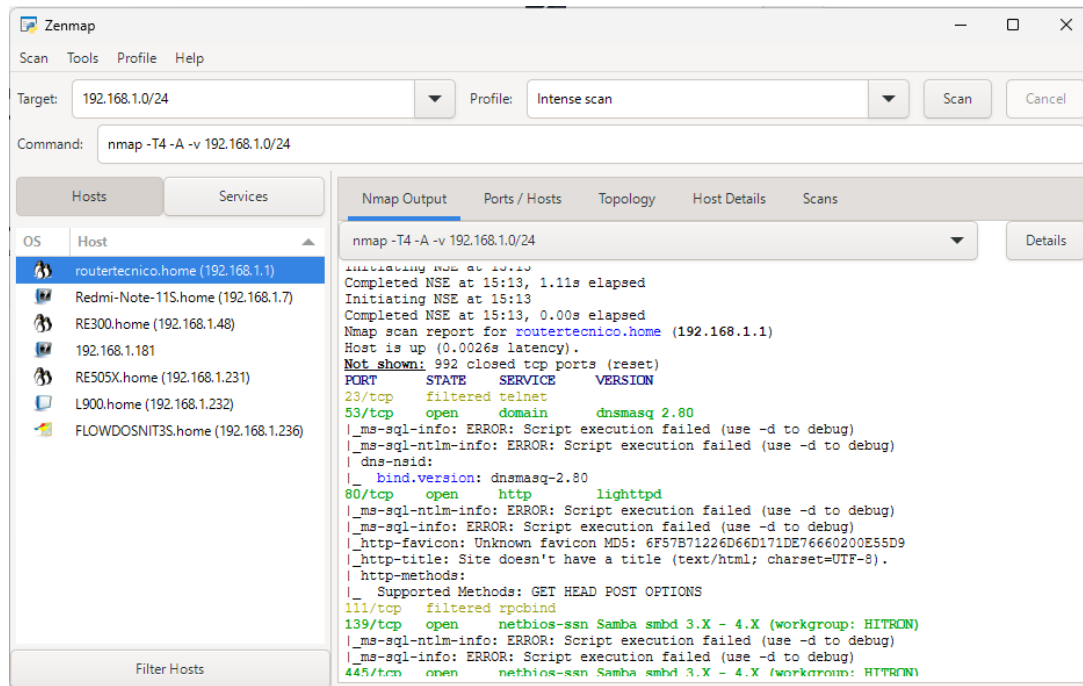


Figura 1.6: Listagem de *hosts* detetados na barra lateral do Zenmap

- **Portas e Serviços:** Acedendo a este separador, é apresentada uma listagem pormenorizada das portas reconhecidas pelo varrimento. Esta secção detalha o número e protocolo da porta, o seu estado, o serviço em execução e a respetiva versão. A informação apresentada é dinâmica, alterando-se consoante a *host* selecionado na barra lateral.

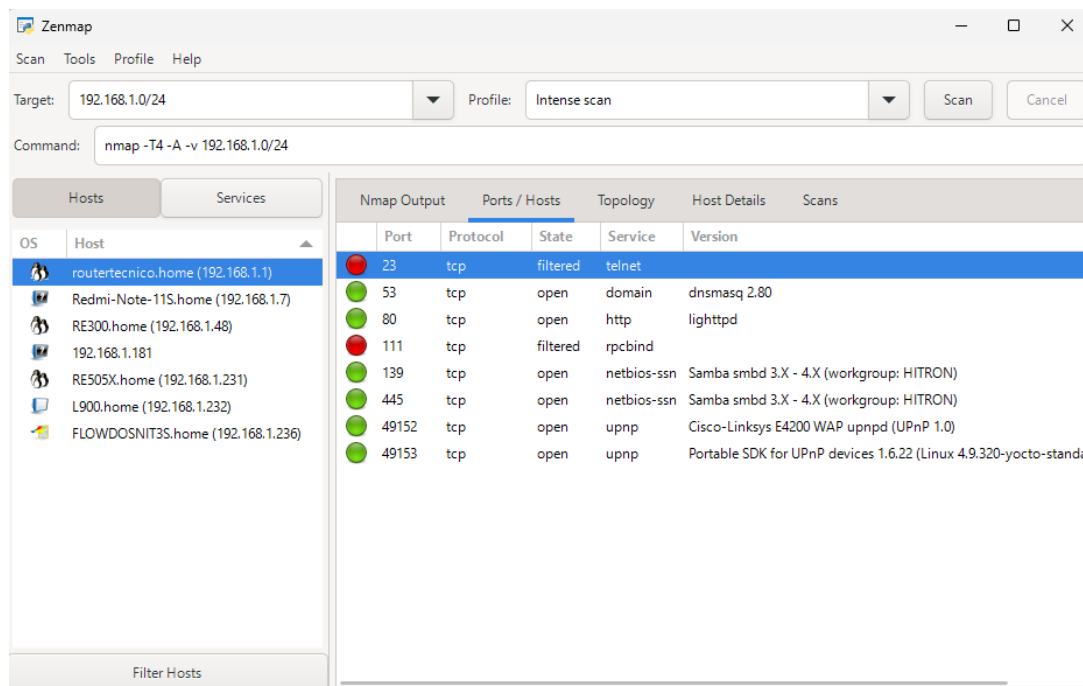


Figura 1.7: Separador *Ports/Hosts* com detalhe das portas, estados e serviços

- **Descrição dos *Hosts*:** Área dedicada à compilação de informações vitais da máquina/dispositivo, tais como o sistema operativo detetado, o endereço MAC, o fabricante da placa de rede e o estado geral do dispositivo.

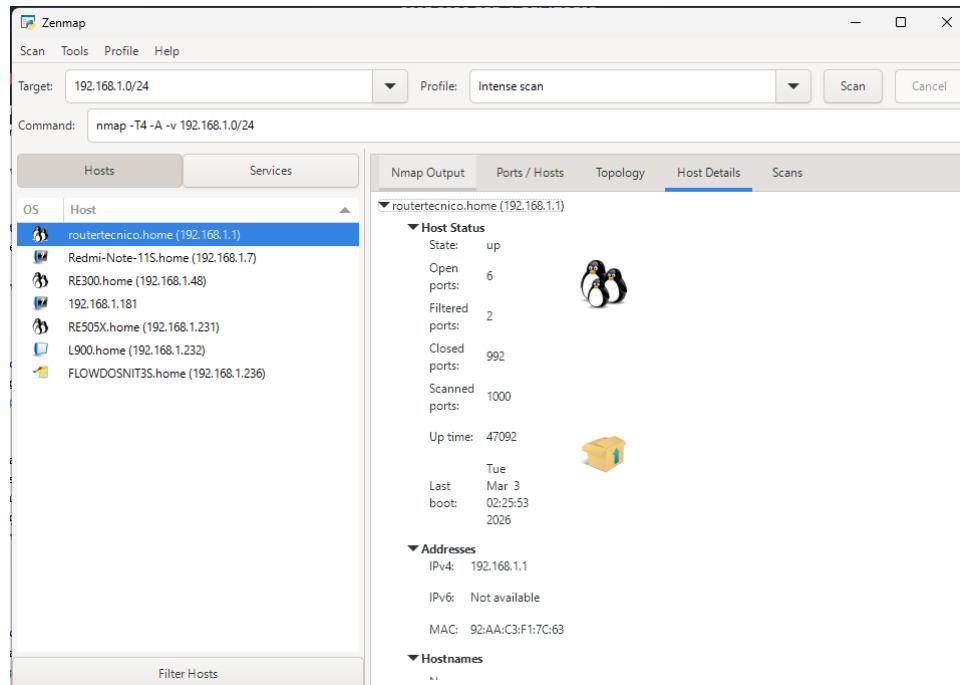


Figura 1.8: Separador *Host Details* com informações vitais do dispositivo

- **Topologia da Rede:** Cumprindo o objetivo de avaliar as aptidões da ferramenta em termos de desenho do mapa da rede, esta funcionalidade ilustra graficamente a estrutura das ligações e a disposição lógica dos *hosts* detetados na infraestrutura.

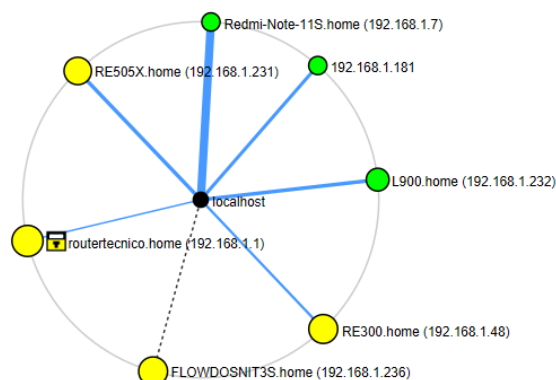


Figura 1.9: Representação gráfica da topologia da rede no Zenmap

1.3 Angry IP Scanner

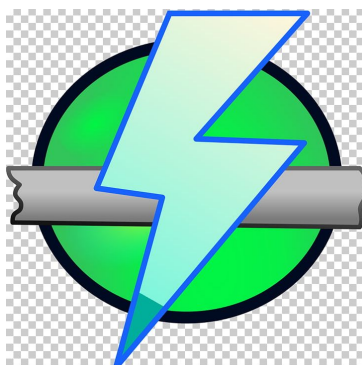


Figura 1.10: Logótipo da aplicação Angry IP Scanner

1.3.1 O que é o Angry IP Scanner?

O **Angry IP Scanner** é uma ferramenta de código aberto amplamente utilizada para a exploração e análise de redes. Permite o varrimento de endereços IP e portas, identificando dispositivos ativos numa rede. A ferramenta é frequentemente utilizada para auditorias de segurança, deteção de dispositivos desconhecidos na rede e *troubleshooting* de conectividade.

Adicionalmente, o Angry IP Scanner oferece a opção de personalizar a forma como os resultados são apresentados. Através do menu *Fetchers*, o utilizador pode seleccionar quais as métricas específicas que pretende visualizar na listagem, tais como as Portas, endereços IP, *Hostnames* e endereços MAC.

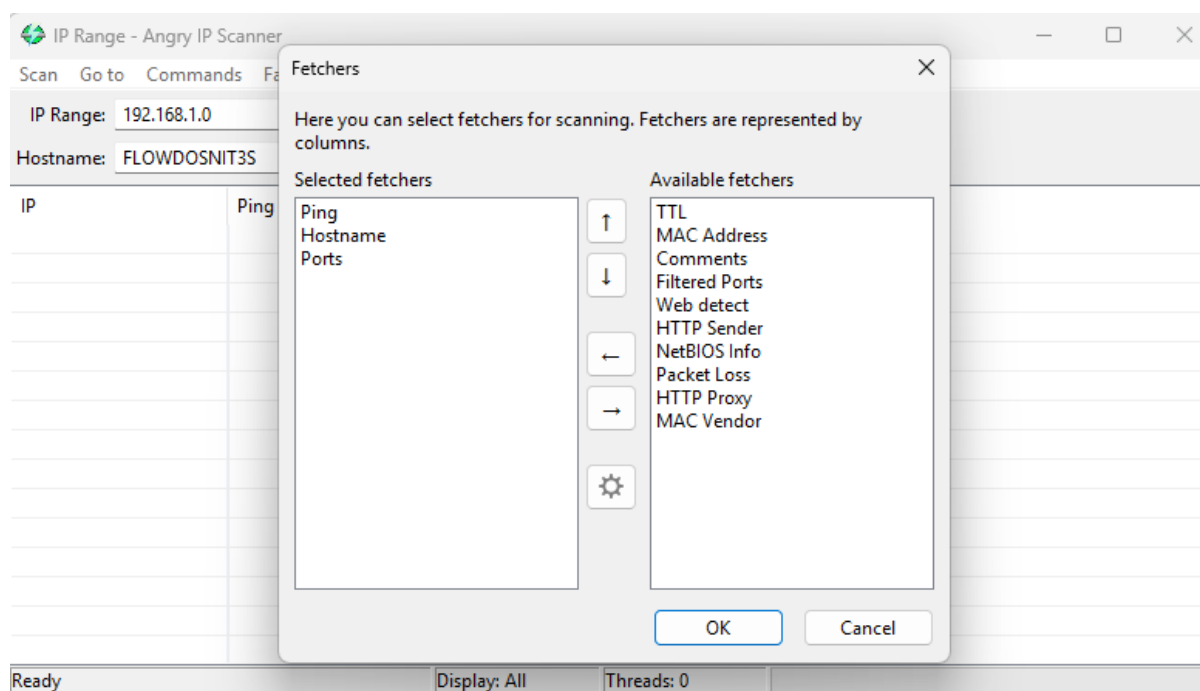


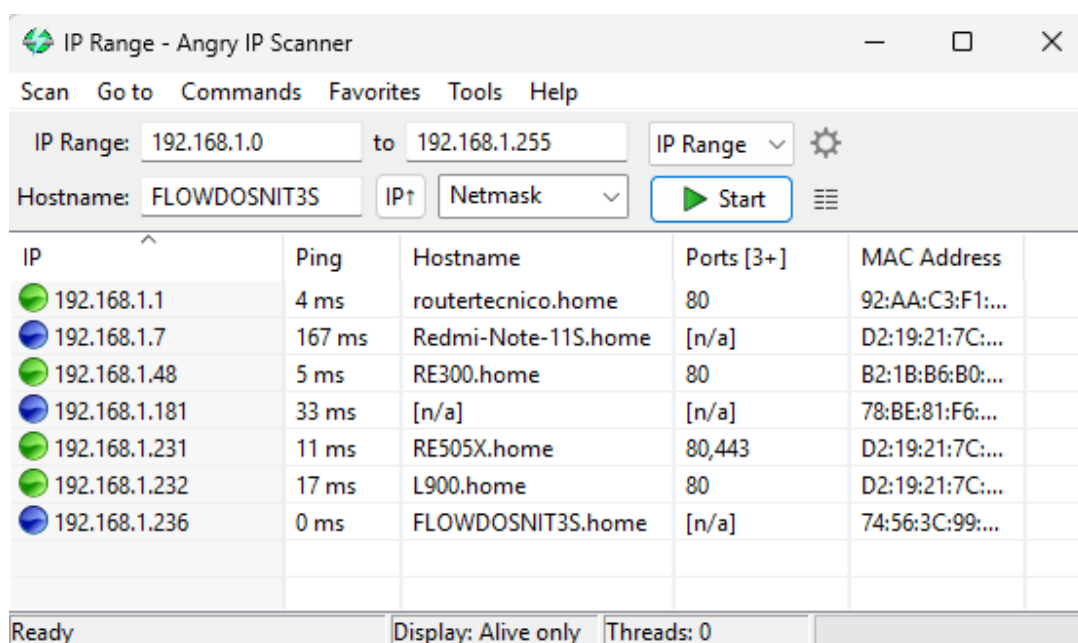
Figura 1.11: Menu de seleção de métricas (Fetchers) no Angry IP Scanner

A aplicação disponibiliza ainda a funcionalidade de restringir o varrimento a um conjunto específico de portas. Esta capacidade de focar a análise apenas nas portas mais relevantes não só elimina o excesso de informação desnecessária, como aumenta substancialmente a velocidade de execução da ferramenta comparativamente a outras alternativas do mercado.

1.3.2 Execução dos Testes

De forma análoga aos testes realizados anteriormente, a análise incidiu sobre a mesma infraestrutura local. Para configurar o varrimento, bastou inserir o primeiro e o último endereço no campo *IP Range*. Em alternativa, a ferramenta permite inserir o primeiro IP da rede e, através da seleção da respetiva máscara de sub-rede, calcular automaticamente o intervalo correto.

Após a correta configuração do intervalo de endereços (**192.168.1.0** a **192.168.1.255**), e definidas as portas e métricas a extrair, iniciou-se o processo clicando no botão *Start*. Aguardou-se o término da verificação rápida da rede para posterior análise dos dispositivos detetados.



The screenshot shows the 'IP Range - Angry IP Scanner' window. The 'IP Range' is set to '192.168.1.0' to '192.168.1.255'. The 'Hostname' is 'FLOWDOSNIT3S'. The 'Start' button is highlighted. Below the input fields is a table of scan results with columns: IP, Ping, Hostname, Ports [3+], and MAC Address. The table lists several active devices with green status indicators and one inactive device with a blue status indicator. At the bottom, there are status bars for 'Ready', 'Display: Alive only', and 'Threads: 0'.

IP	Ping	Hostname	Ports [3+]	MAC Address
192.168.1.1	4 ms	routertecnico.home	80	92:AA:C3:F1:...
192.168.1.7	167 ms	Redmi-Note-11S.home	[n/a]	D2:19:21:7C:...
192.168.1.48	5 ms	RE300.home	80	B2:1B:B6:B0:...
192.168.1.181	33 ms	[n/a]	[n/a]	78:BE:81:F6:...
192.168.1.231	11 ms	RE505X.home	80,443	D2:19:21:7C:...
192.168.1.232	17 ms	L900.home	80	D2:19:21:7C:...
192.168.1.236	0 ms	FLOWDOSNIT3S.home	[n/a]	74:56:3C:99:...

Ready | Display: Alive only | Threads: 0

Figura 1.12: Resultados do varrimento apresentados na interface do Angry IP Scanner

1.3.3 Resultados

O *output* gerado pelo Angry IP Scanner, ilustrado na Figura 1.12, apresenta os resultados de forma tabular e altamente legível. A aplicação utiliza um esquema de cores intuitivo para indicar o estado dos dispositivos detetados: os *hosts* ativos que responderam ao *ping* e têm portas abertas são assinalados com indicadores azuis e verdes, enquanto os endereços inativos são marcados a vermelho.

A tabela permite verificar, de forma imediata, os endereços IP em uso, os respetivos *Hostnames* atribuídos, os endereços MAC e, sempre que possível, o fabricante do *hardware* da placa de rede.

1.4 Conclusão e Comparação das Ferramentas

A execução do **Nmap (Zenmap)** e do **Angry IP Scanner** permitiu avaliar as suas aptidões face ao objetivo proposto neste exercício.

O **Angry IP Scanner** destacou-se pela sua notável rapidez de execução e facilidade de utilização. É a ferramenta ideal para obter um inventário rápido dos dispositivos ligados à rede e verificar a conectividade básica. Contudo, as suas capacidades de desenho topológico e de análise de tráfego profundo são praticamente inexistentes.

Em contrapartida, o **Nmap** revelou ser uma ferramenta significativamente mais poderosa e complexa. Embora exija mais tempo de execução, a sua capacidade de análise de tráfego é exaustiva, identificando sistemas operativos e versões detalhadas de serviços. Além disso, cumpriu de forma exímia o requisito do desenho do mapa da rede, através do seu separador *Topology*, que gerou uma representação visual precisa da infraestrutura analisada.

Em suma, as duas ferramentas revelaram-se complementares: o Angry IP Scanner para varrimentos rápidos de descoberta, e o Nmap para auditorias profundas e mapeamento visual de infraestruturas de rede.

Exercício 2: Ferramentas de Análise de Performance da Rede

2.1 Exercício 2.1

Considere uma rede constituída por dois computadores com OS Linux, conectados diretamente (tenha por base 2 VMs na sua máquina física). Utilize a ferramenta *netperf* para executar alguns testes de performance (TCP e UDP), para tráfego bulk (e.g., FTP, partilha de ficheiros na rede, ...). Considere diferentes tamanhos de pacotes (opção *-m*), para um tempo de teste igual a 60 segundos. O que verifica?

A ferramenta *netperf* permite medir a performance de transmissão de dados utilizando diferentes protocolos de transporte, como o TCP e o UDP, bem como variando parâmetros específicos, como o tamanho dos pacotes.

Neste exercício, foram utilizadas duas máquinas virtuais com o sistema operativo Linux, conectadas diretamente, para analisar a largura de banda (*throughput*) e a taxa de transferência em diferentes cenários. Os testes foram realizados com uma duração fixa de 60 segundos, considerando a simulação de tráfego *bulk* (tráfego volumoso). O objetivo central é identificar como a variação do tamanho dos pacotes afeta a performance global da comunicação na rede.

2.1.1 TCP vs UDP

O protocolo TCP (*Transmission Control Protocol*) é fiável, garante a entrega sequencial dos dados, verifica erros e utiliza mecanismos de controlo de fluxo para evitar a sobrecarga do recetor e o congestionamento da rede. No entanto, apresenta uma velocidade de transmissão inferior devido à sobrecarga introduzida por estes mesmos processos de verificação.

Por outro lado, o protocolo UDP (*User Datagram Protocol*) é consideravelmente mais rápido, porém não fiável. Não garante a entrega nem a ordenação dos pacotes e não possui mecanismos de controlo de fluxo, revelando-se ideal para cenários onde a velocidade é prioritária em relação à fiabilidade.

2.1.2 Preparação das Máquinas Virtuais

Para a realização deste exercício, foram provisionadas duas máquinas virtuais, assumindo uma o papel de servidor e a outra de cliente. Para assegurar a comunicação direta entre ambas, as respetivas interfaces de rede foram configuradas para utilizar a rede interna do *hypervisor*.

A topologia lógica e a comunicação entre as duas máquinas foram estabelecidas da seguinte forma:

- **1ª Máquina (Servidor):**
 - **Nome:** machine1
 - **Endereço IP:** 192.168.1.10
 - **Função:** Servidor (*netserver*)
- **2ª Máquina (Cliente):**
 - **Nome:** machine2
 - **Endereço IP:** 192.168.1.20
 - **Função:** Cliente (*netperf*)

Com base nesta arquitetura, o passo inicial consistiu na atribuição manual dos endereços IP a cada uma das interfaces de rede. Para esse efeito, utilizou-se o seguinte comando no terminal:

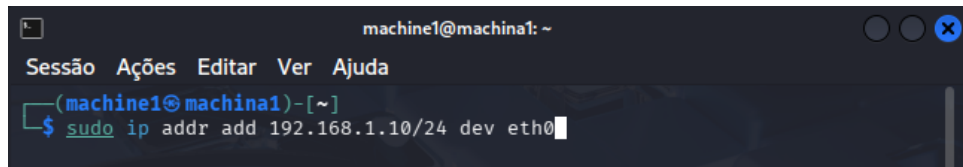
```
sudo ip addr add <IP_da_Máquina>/24 dev <Interface>
```

A anatomia deste comando divide-se nas seguintes componentes essenciais:

- **sudo:** Concede os privilégios de administrador (*root*) necessários para executar o comando e aplicar alterações ao sistema.
- **ip addr add:** Instrução base utilizada para adicionar um novo endereço IP a uma interface de rede.
- **<IP_da_Máquina>:** Representa o endereço IP específico que será configurado (como, por exemplo, 192.168.1.10).
- **dev <Interface>:** Especifica o dispositivo/interface de rede onde o IP será aplicado (como, por exemplo, *eth0*).

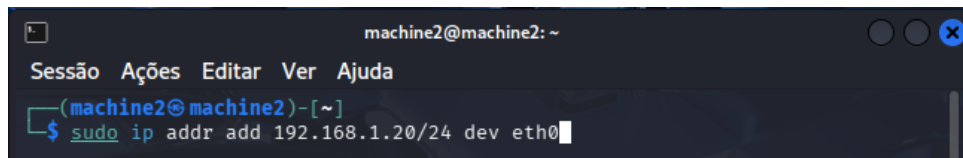
Atribuição dos IPs

Para efetivar esta configuração, o comando de atribuição foi executado de forma sequencial em ambas as máquinas, garantindo que ambas ficassem na mesma rede lógica para a posterior comunicação.



```
machine1@machina1: ~  
Sessão Ações Editar Ver Ajuda  
(machine1@machina1)-[~]  
$ sudo ip addr add 192.168.1.10/24 dev eth0
```

Figura 2.1: Atribuição do endereço IP 192.168.1.10 à máquina servidor

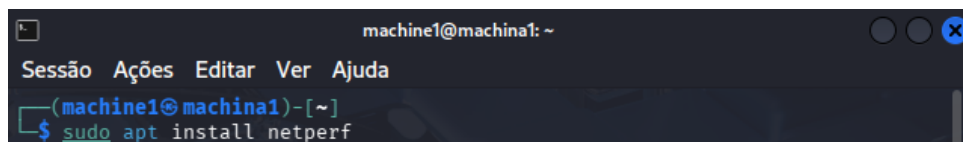


```
machine2@machine2: ~  
Sessão Ações Editar Ver Ajuda  
(machine2@machine2)-[~]  
$ sudo ip addr add 192.168.1.20/24 dev eth0
```

Figura 2.2: Atribuição do endereço IP 192.168.1.20 à máquina cliente

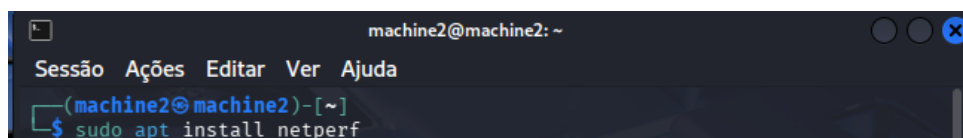
Instalação do Netperf

Após a correta configuração e validação do endereçamento IP em ambas as instâncias, procedeu-se à instalação da ferramenta de testes de performance *netperf*. Esta instalação foi realizada através do gestor de pacotes nativo do Kali Linux, executando o comando `sudo apt install netperf`.



```
machine1@machina1: ~  
Sessão Ações Editar Ver Ajuda  
(machine1@machina1)-[~]  
$ sudo apt install netperf
```

Figura 2.3: Instalação do pacote netperf na máquina servidor

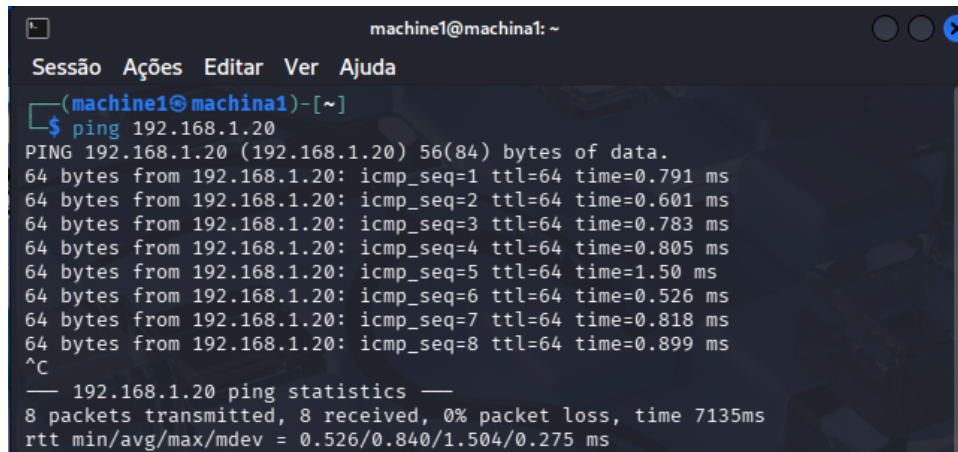


```
machine2@machine2: ~  
Sessão Ações Editar Ver Ajuda  
(machine2@machine2)-[~]  
$ sudo apt install netperf
```

Figura 2.4: Instalação do pacote netperf na máquina cliente

Teste de Conectividade (Ping)

Após a atribuição dos respetivos endereços IP e a instalação das ferramentas necessárias, procedeu-se à validação da conectividade entre as duas instâncias. Para garantir que as máquinas se encontravam na mesma rede lógica e conseguiam comunicar sem restrições de *firewall*, executou-se o comando `ping` a partir de uma das máquinas com destino ao endereço IP da outra.



```
machine1@machina1: ~  
Sessão Ações Editar Ver Ajuda  
(machine1@machina1)-[~]  
$ ping 192.168.1.20  
PING 192.168.1.20 (192.168.1.20) 56(84) bytes of data.  
64 bytes from 192.168.1.20: icmp_seq=1 ttl=64 time=0.791 ms  
64 bytes from 192.168.1.20: icmp_seq=2 ttl=64 time=0.601 ms  
64 bytes from 192.168.1.20: icmp_seq=3 ttl=64 time=0.783 ms  
64 bytes from 192.168.1.20: icmp_seq=4 ttl=64 time=0.805 ms  
64 bytes from 192.168.1.20: icmp_seq=5 ttl=64 time=1.50 ms  
64 bytes from 192.168.1.20: icmp_seq=6 ttl=64 time=0.526 ms  
64 bytes from 192.168.1.20: icmp_seq=7 ttl=64 time=0.818 ms  
64 bytes from 192.168.1.20: icmp_seq=8 ttl=64 time=0.899 ms  
^C  
— 192.168.1.20 ping statistics —  
8 packets transmitted, 8 received, 0% packet loss, time 7135ms  
rtt min/avg/max/mdev = 0.526/0.840/1.504/0.275 ms
```

Figura 2.5: Teste de conectividade (ping) realizado com sucesso entre as instâncias

Confirmada a comunicação bidirecional e a ausência de perda de pacotes neste teste preliminar, avançou-se para a execução dos testes de performance com a ferramenta *netperf*.

2.1.3 Execução dos Testes

Para testar e avaliar a performance da rede, recorreu-se à ferramenta *netperf*, executada através do terminal do Linux. A sintaxe base para a invocação da ferramenta na máquina cliente é a seguinte:

```
netperf -H <IP_do_Servidor> [opções]
```

Para a realização destes ensaios práticos, foram utilizadas as seguintes opções de configuração:

- **-l <tempo>:** Define a duração total do teste, em segundos.
- **-t <teste>:** Especifica o protocolo e o tipo de teste a ser executado
- **-m <tamanho>:** Define o tamanho (em *bytes*) das mensagens/pacotes de dados enviados durante o teste.

Com base nestas opções, os parâmetros globais definidos para os ensaios foram os seguintes:

- **IP do Servidor:** 192.168.1.10 (Máquina 1)
- **Tempo de execução:** 60 segundos
- **Tipos de teste:** Tráfego *bulk* via TCP e UDP
- **Tamanho dos Pacotes:** 1024, 2048 e 4096 *bytes*

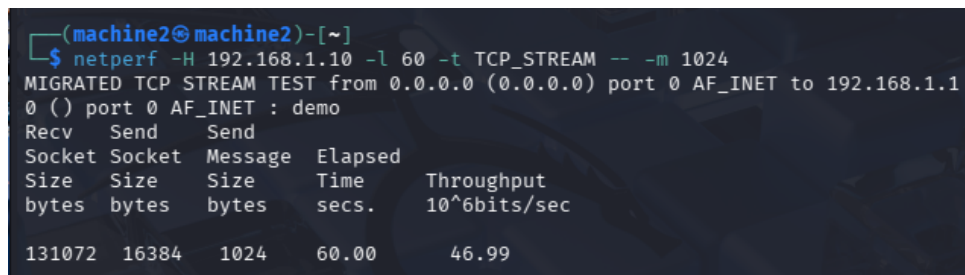
A metodologia adotada consistirá em iniciar os testes com o protocolo TCP, iterando pelos três tamanhos de pacote estipulados. Posteriormente, será executado o mesmo procedimento para o protocolo UDP. Concluída a fase de recolha empírica, proceder-se-á à análise comparativa dos dados de débito (*throughput*) para a formulação das respetivas conclusões.

2.1.4 Testes com Protocolo TCP

Para iniciar a simulação de tráfego volumoso (*bulk*) com fiabilidade de entrega, utilizou-se o protocolo TCP. No terminal do cliente, executaram-se os seguintes comandos de forma sequencial, alterando o tamanho do pacote através do parâmetro `-m`.

Cenário 1: Pacotes de 1024 bytes

Comando executado: `netperf -H 192.168.1.10 -l 60 -t TCP_STREAM -- -m 1024`

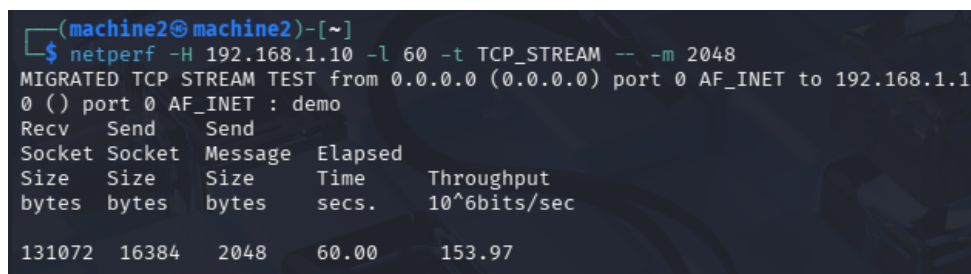


```
(machine2@machine2)-[~]
$ netperf -H 192.168.1.10 -l 60 -t TCP_STREAM -- -m 1024
MIGRATED TCP STREAM TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 192.168.1.1
0 () port 0 AF_INET : demo
Recv  Send  Send
Socket Socket Message Elapsed
Size  Size  Size  Time  Throughput
bytes bytes bytes secs.  10^6bits/sec
131072 16384 1024 60.00 46.99
```

Figura 2.6: Resultado do teste TCP_STREAM com pacotes de 1024 bytes

Cenário 2: Pacotes de 2048 bytes

Comando executado: `netperf -H 192.168.1.10 -l 60 -t TCP_STREAM -- -m 2048`

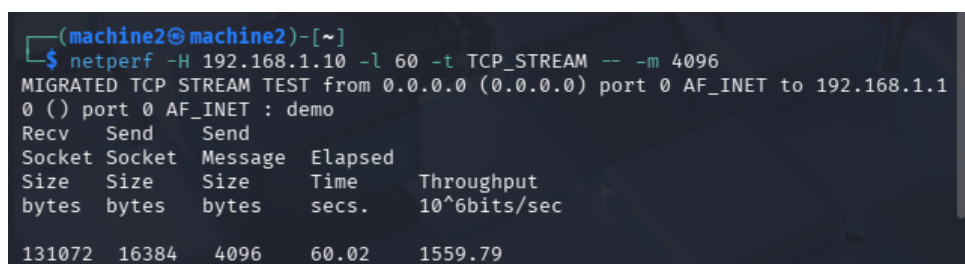


```
(machine2@machine2)-[~]
$ netperf -H 192.168.1.10 -l 60 -t TCP_STREAM -- -m 2048
MIGRATED TCP STREAM TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 192.168.1.1
0 () port 0 AF_INET : demo
Recv  Send  Send
Socket Socket Message Elapsed
Size  Size  Size  Time  Throughput
bytes bytes bytes secs.  10^6bits/sec
131072 16384 2048 60.00 153.97
```

Figura 2.7: Resultado do teste TCP_STREAM com pacotes de 2048 bytes

Cenário 3: Pacotes de 4096 bytes

Comando executado: `netperf -H 192.168.1.10 -l 60 -t TCP_STREAM -- -m 4096`



```
(machine2@machine2)-[~]
$ netperf -H 192.168.1.10 -l 60 -t TCP_STREAM -- -m 4096
MIGRATED TCP STREAM TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 192.168.1.1
0 () port 0 AF_INET : demo
Recv  Send  Send
Socket Socket Message Elapsed
Size  Size  Size  Time  Throughput
bytes bytes bytes secs.  10^6bits/sec
131072 16384 4096 60.02 1559.79
```

Figura 2.8: Resultado do teste TCP_STREAM com pacotes de 4096 bytes

2.1.5 Observação dos Resultados

A partir dos testes TCP realizados e ilustrados nas figuras anteriores, foi possível compilar os dados de performance da ligação. Em todos os cenários, os tamanhos dos *buffers* do *socket* mantiveram-se consistentes com a configuração padrão do sistema operativo:

- Tamanho do *Buffer* de Receção : 131072 bytes
- Tamanho do *Buffer* de Envio : 16384 bytes

A Tabela 2.1 sintetiza os resultados do débito (*throughput*) obtido em função da variação do tamanho da mensagem configurada para cada ensaio prático.

Tamanho do Pacote (<i>bytes</i>)	<i>Throughput</i> (10^6 bits/sec)
1024	46.99
2048	153.97
4096	1559.79

Tabela 2.1: Resultados do *throughput* TCP em função do tamanho do pacote

2.1.6 Testes com Protocolo UDP

De forma análoga aos testes TCP, procedeu-se à avaliação da rede utilizando o protocolo UDP (parâmetro `-t UDP_STREAM`). Pela natureza não fiável deste protocolo, a ferramenta *netperf* apresenta duas linhas de resultados em cada teste: a primeira reflete o tráfego gerado e enviado pela máquina local, e a segunda reflete o tráfego efetivamente recebido pela máquina remota.

Cenário 1: Pacotes de 1024 bytes

Comando executado: `netperf -H 192.168.1.10 -l 60 -t UDP_STREAM -- -m 1024`

```
(machine2@machine2)-[~]
$ netperf -H 192.168.1.10 -l 60 -t UDP_STREAM -- -m 1024
MIGRATED UDP STREAM TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 192.168.1.1
0 () port 0 AF_INET : demo
Socket  Message  Elapsed      Messages
Size   Size    Time         Okay Errors   Throughput
bytes  bytes   secs          #      #      10^6bits/sec
212992   1024   60.00      249750      0       34.10
212992           60.00      219870           30.02
```

Figura 2.9: Resultado do teste UDP_STREAM com pacotes de 1024 bytes

Cenário 2: Pacotes de 2048 bytes

Comando executado: `netperf -H 192.168.1.10 -l 60 -t UDP_STREAM -- -m 2048`


```
(machine2@machine2)-[~]
$ netperf -H 192.168.1.10 -l 60 -t UDP_STREAM -- -m 2048
MIGRATED UDP STREAM TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 192.168.1.1
0 () port 0 AF_INET : demo
Socket Message Elapsed      Messages
Size   Size   Time      Okay Errors  Throughput
bytes  bytes  secs      #      #      10^6bits/sec

212992    2048   60.01    122983     0     33.58
212992           60.01    111462           30.43
```

Figura 2.10: Resultado do teste UDP_STREAM com pacotes de 2048 bytes

Cenário 3: Pacotes de 4096 bytes

Comando executado: `netperf -H 192.168.1.10 -l 60 -t UDP_STREAM -- -m 4096`

```
(machine2@machine2)-[~]
$ netperf -H 192.168.1.10 -l 60 -t UDP_STREAM -- -m 4096
MIGRATED UDP STREAM TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 192.168.1.1
0 () port 0 AF_INET : demo
Socket Message Elapsed      Messages
Size   Size   Time      Okay Errors  Throughput
bytes  bytes  secs      #      #      10^6bits/sec

212992    4096   60.00     85599     0     46.75
212992           60.00     80082           43.73
```

Figura 2.11: Resultado do teste UDP_STREAM com pacotes de 4096 bytes

2.1.7 Observação e Análise

A Tabela 2.2 compila os valores de *throughput* registados durante os testes UDP, distinguindo o tráfego enviado do tráfego efetivamente recebido com sucesso. Verificou-se que o tamanho do *Socket Size* utilizado pelo sistema foi de 212992 *bytes*.

Tamanho do Pacote (<i>bytes</i>)	Enviado (10^6 bits/sec)	Recebido (10^6 bits/sec)
1024	34.10	30.02
2048	33.58	30.43
4096	46.75	43.73

Tabela 2.2: Resultados do *throughput* UDP em função do tamanho do pacote

2.1.8 Comparação dos Testes e Conclusão Final

A análise comparativa entre os ensaios realizados permitiu extrair conclusões cruciais sobre o comportamento dos dois protocolos no transporte de tráfego volumoso (*bulk*):

- **Performance Global (TCP vs. UDP):** Nos presentes testes em ambiente virtualizado, o **TCP** apresentou uma taxa de transferência substancialmente superior à do UDP. Embora o

UDP seja teoricamente mais rápido devido à ausência de garantias de entrega, em infraestruturas virtuais, o protocolo TCP beneficia largamente de otimizações modernas do sistema operativo. Estas otimizações permitem ao processador copiar blocos gigantes de dados na memória de forma altamente eficiente, algo que o UDP não consegue acompanhar com a mesma fluidez.

- **Impacto do Tamanho da Mensagem:** O parâmetro do tamanho do pacote (`-m`) afeta a performance de forma direta e distinta:
 - **No TCP:** Um tamanho reduzido (1024 *bytes*) causa uma enorme sobrecarga (*overhead*) de processamento de cabeçalhos face à carga útil, limitando a velocidade. Pelo contrário, o tamanho de **4096 *bytes* revelou-se o mais eficiente**, disparando o *throughput* para cerca de 1.5 Gbps.
 - **No UDP:** A adoção de pacotes maiores também resulta, regra geral, num *throughput* mais elevado (como se verificou no salto para os 4096 *bytes*), desde que os *buffers* do sistema consigam suportar a carga sem perdas excessivas.
- **Adequabilidade de Uso:** O **TCP** consolida-se como o protocolo ideal para transmissões que exigem fiabilidade absoluta e integridade dos dados, tais como transferências de ficheiros e navegação *web*. Em contrapartida, o **UDP** mantém-se mais eficiente e indicado para aplicações em tempo real (como chamadas de voz, *streaming* de vídeo ou jogos *online*), onde o atraso (*delay*) é crítico e a perda de pacotes pontuais não compromete a experiência global.

2.2 Exercício 2.2

Repita o exercício anterior, mas agora recorrendo a uma VM de um colega seu (ou seja, teste entre uma VM no seu computador, e uma VM no computador do seu colega). Comece por usar a infraestrutura cablada da rede. O que verifica? Procure informações sobre o uso da opção `-m`". Esta opção permitiria encontrar que problema?

Para a resolução deste exercício, seria necessária a utilização de uma infraestrutura de rede física com ligação por cabo (Ethernet). No entanto, não foi possível realizar esta componente prática, uma vez que o equipamento *host* utilizado não possui uma interface de rede física (porta RJ45) para ligação cablada.

Dado que a ligação física direta não pôde ser estabelecida, não foi possível registar o comportamento empírico esperado para este tipo de infraestrutura.

Relativamente à questão teórica sobre a opção `-m` (que define o tamanho da mensagem/pacote a enviar), a sua manipulação metódica permite identificar problemas relacionados com a **fragmentação de pacotes** e descobrir o **MTU (*Maximum Transmission Unit*)** do caminho da rede. Se o tamanho do pacote enviado for superior ao MTU suportado por qualquer equipamento intermédio (como *switches* ou *routers*), o pacote terá de ser fragmentado ou, caso a fragmentação não seja permitida, será descartado. Esta opção é, portanto, vital para diagnosticar quedas abruptas de desempenho e estrangulamentos na comunicação causados por pacotes excessivamente grandes.

2.3 Exercício 2.3

Repita o exercício anterior, mas agora usando a rede sem fios. Compare os resultados com aqueles obtidos na alínea anterior. Que verifica?

Este exercício tem como intuito replicar a comunicação entre duas máquinas virtuais distintas (alojadas em computadores físicos diferentes), utilizando, desta vez, uma rede sem fios (Wi-Fi) em vez da infraestrutura cablada.

Devido à impossibilidade técnica de realizar o Exercício 2.2, a análise comparativa incidirá sobre os resultados obtidos neste ensaio (Wi-Fi) em contraste com os resultados do Exercício 2.1 (Rede Interna virtual). Esta abordagem permitirá evidenciar as diferenças de performance entre um ambiente de rede simulado e isolado e uma transmissão real efetuada através do ar.

A utilização de uma rede sem fios introduz variáveis externas que influenciam diretamente a latência, o *throughput* e a estabilidade da conexão. Fatores como a interferência de radiofrequência, a distância ao ponto de acesso e o congestionamento do meio partilhado tornam os resultados intrinsecamente voláteis. Consciente de que uma rede Wi-Fi não oferece um ambiente 100% estável e isolado, optou-se por executar os testes múltiplas vezes. O objetivo primordial não consiste em registar valores exatos e absolutos, mas sim em identificar padrões de comportamento consistentes face à variação do protocolo e do tamanho dos pacotes.

2.3.1 Preparação das Máquinas na Rede Sem Fios

Para a execução deste cenário, utilizaram-se dois computadores físicos distintos, cada um a executar uma máquina virtual, ambos conectados à mesma rede Wi-Fi local.

O primeiro passo consistiu na reconfiguração das interfaces de rede das VMs. Nas definições do *hypervisor*, o modo de rede foi alterado de "Rede Interna" para "**Adaptador Bridge**" (*Bridged Adapter*). Esta configuração permite que as placas de rede virtuais contornem o NAT do sistema *host* e solicitem um endereço IP diretamente ao servidor DHCP do *router* local, passando a operar como dispositivos independentes na rede física.

Uma vez aplicada esta configuração, executou-se o comando `ip a` no terminal de cada VM para verificar a atribuição correta dos novos endereços IP na sub-rede sem fios, garantindo assim os pressupostos necessários para a comunicação direta entre as máquinas dos dois colegas.

```
(machine1@machina1)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:a6:0e:54 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.250/24 brd 192.168.1.255 scope global dynamic noprefixroute eth0
        valid_lft 2980sec preferred_lft 2980sec
    inet6 fe80::a00:27ff:fea6:e54/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Figura 2.12: Verificação do endereço IP atribuído à máquina(Servidor)

```
(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:d1:f8:5d brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.252/24 brd 192.168.1.255 scope global dynamic noprefixroute eth0
        valid_lft 3477sec preferred_lft 3477sec
    inet6 fe80::efde:1520:9e87:8c00/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Figura 2.13: Verificação do endereço IP atribuído à máquina(Cliente)

Como resultado da atribuição dinâmica efetuada pelo *router*, a topologia lógica de testes ficou estruturada da seguinte forma:

- **1ª Máquina (Servidor):**
 - Nome: machine1
 - Proprietário: Autor do Relatório
 - Endereço IP: 192.168.1.250
 - Função: Servidor
- **2ª Máquina (Cliente):**
 - Nome: kali
 - Proprietário: Autor do Relatório
 - Endereço IP: 192.168.1.252
 - Função: Cliente

Para confirmar de forma inequívoca que ambas as instâncias conseguiam comunicar entre si através da infraestrutura sem fios, procedeu-se à validação da conectividade bidirecional executando um teste com o comando `ping`.

```
(kali@kali)-[~]
$ ping 192.168.1.250
PING 192.168.1.250 (192.168.1.250) 56(84) bytes of data.
64 bytes from 192.168.1.250: icmp_seq=1 ttl=64 time=167 ms
64 bytes from 192.168.1.250: icmp_seq=2 ttl=64 time=11.8 ms
64 bytes from 192.168.1.250: icmp_seq=3 ttl=64 time=17.6 ms
64 bytes from 192.168.1.250: icmp_seq=4 ttl=64 time=14.4 ms
64 bytes from 192.168.1.250: icmp_seq=5 ttl=64 time=13.8 ms
64 bytes from 192.168.1.250: icmp_seq=6 ttl=64 time=29.6 ms
64 bytes from 192.168.1.250: icmp_seq=7 ttl=64 time=13.9 ms
64 bytes from 192.168.1.250: icmp_seq=8 ttl=64 time=30.3 ms
^C
— 192.168.1.250 ping statistics —
8 packets transmitted, 8 received, 0% packet loss, time 7012ms
rtt min/avg/max/mdev = 11.819/37.258/166.618/49.361 ms
```

Figura 2.14: Validação da conectividade bidirecional (ping) entre as instâncias via rede Wi-Fi

2.3.2 Execução dos Testes de Rede Sem Fios

Com a confirmação do sucesso na comunicação via rede sem fios, avançou-se para a fase de medição de performance. A metodologia e a sintaxe dos comandos executados através da ferramenta *netperf* seguiram rigorosamente a mesma estrutura detalhada no Exercício 2.1, dispensando nova explicação exaustiva dos seus parâmetros.

Para esta nova iteração em ambiente Wi-Fi, os parâmetros globais estabelecidos foram os seguintes:

- **IP do Servidor:** 192.168.1.250 (Máquina 1 - Servidor)
- **Tempo de execução:** 60 segundos por teste
- **Tipos de teste:** Simulação de tráfego via protocolos TCP e UDP
- **Tamanho dos Pacotes (-m):** 1024, 2048 e 4096 *bytes*

2.3.3 Testes com Protocolo TCP

Para avaliar a performance do protocolo TCP na rede sem fios, executaram-se os seguintes comandos no terminal do cliente, alterando progressivamente o tamanho do pacote através do parâmetro *-m*.

Cenário 1: Pacotes de 1024 bytes

Comando executado: `netperf -H 192.168.1.250 -l 60 -t TCP_STREAM -- -m 1024`

```
(kali@kali)-[~]
$ netperf -H 192.168.1.250 -l 60 -t TCP_STREAM -- -m 1024
MIGRATED TCP STREAM TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 192.168.1.250 () port 0 AF_INET
ET : demo
Recv  Send  Send
Socket Socket Message Elapsed
Size  Size  Size  Time   Throughput
bytes bytes bytes secs.  10^6bits/sec
131072 16384 1024 61.15 29.37
```

Figura 2.15: Resultado do teste TCP via Wi-Fi com pacotes de 1024 bytes

Cenário 2: Pacotes de 2048 bytes

Comando executado: `netperf -H 192.168.1.250 -l 60 -t TCP_STREAM -- -m 2048`

```
(kali@kali)-[~]
$ netperf -H 192.168.1.250 -l 60 -t TCP_STREAM -- -m 2048
MIGRATED TCP STREAM TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 192.168.1.250 () port 0 AF_IN
ET : demo
Recv  Send  Send
Socket Socket Message Elapsed
Size  Size  Size  Time  Throughput
bytes bytes bytes secs.  10^6bits/sec
131072 16384 2048 60.85 29.33
```

Figura 2.16: Resultado do teste TCP via Wi-Fi com pacotes de 2048 bytes

Cenário 3: Pacotes de 4096 bytes

Comando executado: `netperf -H 192.168.1.250 -l 60 -t TCP_STREAM -- -m 4096`

```
(kali@kali)-[~]
$ netperf -H 192.168.1.250 -l 60 -t TCP_STREAM -- -m 4096
MIGRATED TCP STREAM TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 192.168.1.250 () port 0 AF_IN
ET : demo
Recv  Send  Send
Socket Socket Message Elapsed
Size  Size  Size  Time  Throughput
bytes bytes bytes secs.  10^6bits/sec
131072 16384 4096 60.95 28.93
```

Figura 2.17: Resultado do teste TCP via Wi-Fi com pacotes de 4096 bytes

2.3.4 Testes com Protocolo UDP

De seguida, repetiu-se a mesma metodologia para o protocolo UDP, utilizando os comandos descritos abaixo para gerar tráfego não fiável na infraestrutura sem fios.

Cenário 1: Pacotes de 1024 bytes

Comando executado: `netperf -H 192.168.1.250 -l 60 -t UDP_STREAM -- -m 1024`

```
(kali@kali)-[~]
$ netperf -H 192.168.1.250 -l 60 -t UDP_STREAM -- -m 1024
MIGRATED UDP STREAM TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 192.168.1.250 () port 0 AF_IN
ET : demo
send_data: data send error: No route to host (errno 113)
netperf: send_omni: send_data failed: No route to host

(kali@kali)-[~]
$ netperf -H 192.168.1.250 -l 60 -t UDP_STREAM -- -m 1024
MIGRATED UDP STREAM TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 192.168.1.250 () port 0 AF_IN
ET : demo
send_data: data send error: No route to host (errno 113)
netperf: send_omni: send_data failed: No route to host
```

Figura 2.18: Resultado do teste UDP via Wi-Fi com pacotes de 1024 bytes

Cenário 2: Pacotes de 2048 bytes

Comando executado: `netperf -H 192.168.1.250 -l 60 -t UDP_STREAM -- -m 2048`

```
(kali@kali)-[~]
└─$ netperf -H 192.168.1.250 -l 60 -t UDP_STREAM -- -m 2048
MIGRATED UDP STREAM TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 192.168.1.250 () port 0 AF_INET
ET : demo
Socket  Message  Elapsed    Messages
Size    Size    Time      Okay Errors  Throughput
bytes   bytes   secs          #      #      10^6bits/sec

212992    2048   60.00    1562890    0    426.77
212992    60.00    72523    19.80
```

Figura 2.19: Resultado do teste UDP via Wi-Fi com pacotes de 2048 bytes

Cenário 3: Pacotes de 4096 bytes

Comando executado: `netperf -H 192.168.1.250 -l 60 -t UDP_STREAM -- -m 4096`

```
(kali@kali)-[~]
└─$ netperf -H 192.168.1.250 -l 60 -t UDP_STREAM -- -m 4096
MIGRATED UDP STREAM TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 192.168.1.250 () port 0 AF_INET
ET : demo
Socket  Message  Elapsed    Messages
Size    Size    Time      Okay Errors  Throughput
bytes   bytes   secs          #      #      10^6bits/sec

212992    4096   60.00    787727    0    430.20
212992    60.00    26246    14.33
```

Figura 2.20: Resultado do teste UDP via Wi-Fi com pacotes de 4096 bytes

2.3.5 Observação dos Resultados Wi-Fi

A partir das capturas efetuadas, compilaram-se os valores de débito (*throughput*) na rede sem fios. Em conformidade com a alocação padrão do sistema operativo, registaram-se os seguintes tamanhos de memória para as ligações TCP:

- **Tamanho do *Buffer* de Receção:** 131072 *bytes*
- **Tamanho do *Buffer* de Envio:** 16384 *bytes*

A Tabela 2.3 demonstra os resultados obtidos para o protocolo TCP.

Tamanho do Pacote (<i>bytes</i>)	<i>Throughput</i> TCP (10 ⁶ bits/sec)
1024	29.37
2048	29.33
4096	28.93

Tabela 2.3: Resultados do *throughput* TCP via Wi-Fi

No que diz respeito aos testes UDP, o tamanho do *Buffer* de Envio alocado pelo sistema foi de 212992 *bytes*.

É imperativo referir que, durante a execução do ensaio UDP com pacotes de 1024 *bytes*, o sistema reportou consecutivamente um erro de encaminhamento. Após a execução de múltiplas repetições do

teste, o erro manteve-se consistente. Este comportamento atesta uma falha na capacidade de escoamento do *router* doméstico ou da placa de rede sem fios, que colapsou face à injeção abrupta e massiva de dezenas de milhares de pequenos datagramas por segundo, bloqueando a rota para o destino. Devido a esta instabilidade estrutural, a Tabela 2.4 reflete o comportamento do protocolo UDP apenas para os pacotes de 2048 e 4096 *bytes*.

Tamanho do Pacote (<i>bytes</i>)	UDP Enviado (Mbps)	UDP Recebido (Mbps)
2048	426.77	19.80
4096	430.20	14.33

Tabela 2.4: Resultados do *throughput* UDP via Wi-Fi

2.3.6 Conclusão Parcial: Análise Exclusiva da Rede Wi-Fi

A análise isolada dos ensaios realizados na infraestrutura sem fios permite extrair duas conclusões fundamentais sobre o comportamento dos protocolos num meio de transmissão físico partilhado e suscetível a constrangimentos:

- **No protocolo TCP:** Observou-se que a alteração do tamanho do pacote (de 1024 para 4096 *bytes*) não produziu qualquer variação significativa no *throughput*, estabilizando sempre de forma muito constante em torno dos 29 Mbps. Isto indica que a velocidade máxima não foi limitada pelo tempo de processamento dos pacotes, mas sim pela barreira física da própria placa Wi-Fi e do *router*. Graças ao seu mecanismo de controlo de fluxo, o TCP respeitou esse limite e garantiu uma transmissão sem perdas.
- **No protocolo UDP:** Os resultados evidenciaram uma sobrecarga severa da infraestrutura. A máquina cliente gerou e tentou enviar tráfego à sua velocidade máxima de processamento (cerca de 430 Mbps), mas a rede sem fios não teve capacidade para escoar esse volume pelo ar. Pela ausência de controlo de fluxo no UDP, o excesso de pacotes resultou num estrangulamento no *router*, originando taxas de receção reais muito baixas (14 a 19 Mbps), uma perda de pacotes massiva e falhas de encaminhamento quando se tentou enviar um volume demasiado elevado de pacotes pequenos (1024 *bytes*).

2.3.7 Comparação de Resultados: Wi-Fi vs. Rede Interna Virtual (Ex. 2.1)

A transição de um ambiente puramente virtualizado (Exercício 2.1) para uma transmissão real através de uma infraestrutura Wi-Fi revelou diferenças drásticas de desempenho, constituindo um caso de estudo prático sobre o comportamento dos protocolos em redes limitadas fisicamente:

- **Estrangulamento (*Bottleneck*) Físico:** No Exercício 2.1, o protocolo TCP escalou até aos impressionantes 1559.79 Mbps, uma vez que a comutação ocorreu na memória RAM. Na rede Wi-Fi, o *throughput* TCP estagnou categoricamente na casa dos ~29 Mbps em todos os ensaios,

anulando a vantagem teórica de aumentar o tamanho do pacote ($-m$). Isto comprova que o meio aéreo (a capacidade física da norma 802.11 e as interferências do ambiente) assumiu o papel de gargalo da rede, limitando o débito máximo independentemente das otimizações na camada de transporte.

- **Eficácia do Controlo de Fluxo (TCP):** Os resultados demonstram de forma cabal o mecanismo de Controlo de Fluxo e Prevenção de Congestionamento do TCP. A máquina emissora percecionou a limitação do canal Wi-Fi e "acelerou" apenas até à velocidade máxima que a rede conseguia suportar (~ 29 Mbps), garantindo a entrega fiável e sem perda de pacotes.
- **Saturação da Rede no UDP:** Por outro lado, o teste UDP espelha a ausência de mecanismos de controlo. A máquina cliente gerou e injetou tráfego no adaptador sem fios a mais de 426 Mbps. Contudo, como o *router* Wi-Fi e a interface de receção não possuíam capacidade para escoar este volume de dados pelo ar, a esmagadora maioria dos pacotes foi descartada silenciosamente. Isto resultou numa taxa de receção pífia de apenas 14 a 19 Mbps e numa taxa de perda de pacotes (*packet loss*) superior a 95%.

Conclusão: Comprova-se que a infraestrutura física dita os limites absolutos de transmissão. Numa rede sem fios, o TCP atua de forma adaptativa para garantir a fiabilidade, enquanto o tráfego *bulk* em UDP satura a ligação, resultando em quedas severas de desempenho e ineficiência do meio partilhado.

2.4 Exercício 2.4

Execute os testes efetuados de 2.1) a 2.3) mas para tráfego transacional (e.g., base de dados, ...). Neste tipo de conexões, cliente-servidor com transações de dados, o cliente envia pequenos pacotes com consultas de informação, e o servidor processa-as e retorna a resposta, consistindo geralmente em dados de elevada dimensão. Dica: use TCP_RR, e experimente valores para o parâmetro “-r”. Compare os resultados com UDP_RR.

Contrariamente aos exercícios anteriores, que avaliavam a capacidade da rede no transporte de fluxos contínuos e volumosos de dados, este exercício foca-se no desempenho da rede em cenários de **tráfego transacional**. Neste tipo de comunicação, típico de consultas a bases de dados ou pedidos HTTP isolados, a métrica de avaliação principal deixa de ser o débito em Megabits por segundo (Mbps) e passa a ser o número de **Transações Por Segundo (TPS)** que o sistema consegue concluir.

2.4.1 Metodologia

Para a simulação de tráfego de pedido/resposta (*Request/Response*), recorreu-se aos modos TCP_RR e UDP_RR da ferramenta *netperf*.

A sintaxe base do comando foi adaptada para incluir um novo parâmetro de dimensionamento:

```
netperf -H <IP_Servidor> -t <Protocolo> -l <Tempo> -- -r <Req>,<Resp>
```

A anatomia deste comando difere dos ensaios anteriores devido à introdução da *flag* `-r`, que permite definir especificamente o tamanho do pedido (*Request*) enviado pelo cliente e o tamanho da resposta (*Response*) devolvida pelo servidor:

- `-r 1,1`: Pedido e Resposta pequena (simula um "ping").
- `-r 1,1024`: Pedido pequeno e Resposta Grande (simula uma consulta SQL que retorna muitos dados).
- `-r 1024,1024`: Pedido e Resposta grande (simula uma transferência de dados).

Os testes foram configurados para uma duração de **60 segundos** cada.

2.4.2 Topologia de Teste (Rede Interna Virtual)

Para iniciar a recolha de resultados transacionais e estabelecer uma *baseline* (linha de base) livre de interferências externas, reativou-se a topologia da **Rede Interna** (*Internal Network*) configurada e validada no Exercício 2.1.

A arquitetura lógica retomou os seguintes parâmetros:

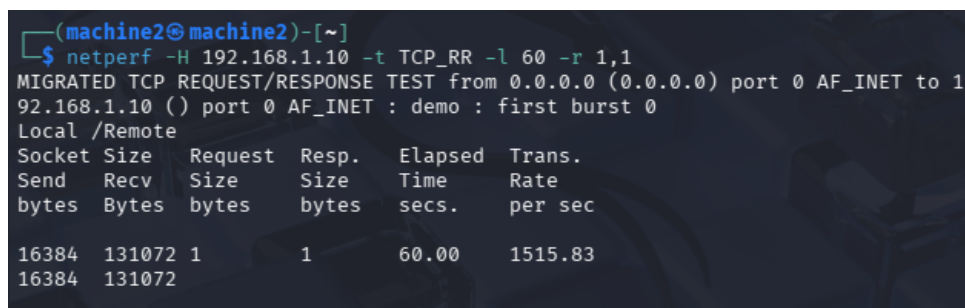
- **1ª Máquina (Servidor)**: machine1 — IP: 192.168.1.10 — Função: Servidor
- **2ª Máquina (Cliente)**: machine2 — IP: 192.168.1.20 — Função: Cliente

2.4.3 Testes com Protocolo TCP

A avaliação do tráfego transacional fiável iniciou-se com o protocolo TCP. Alterando os parâmetros da opção `-r`, executaram-se os testes para aferir o número de transações por segundo suportadas pela infraestrutura virtual.

Cenário 1: Pedido de 1 byte / Resposta de 1 byte

Comando executado: `netperf -H 192.168.1.10 -t TCP_RR -l 60 -r 1,1`



```
(machine2@machine2)-[~]
$ netperf -H 192.168.1.10 -t TCP_RR -l 60 -r 1,1
MIGRATED TCP REQUEST/RESPONSE TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 1
92.168.1.10 () port 0 AF_INET : demo : first burst 0
Local /Remote
Socket Size  Request  Resp.   Elapsed  Trans.
Send  Recv  Size    Size    Time    Rate
bytes Bytes bytes   bytes   secs.   per sec
16384 131072 1        1       60.00   1515.83
16384 131072
```

Figura 2.21: Resultado do teste transacional TCP_RR (1 byte pedido / 1 byte resposta)

Cenário 2: Pedido de 1 byte / Resposta de 1024 bytes

Comando executado: `netperf -H 192.168.1.10 -t TCP_RR -l 60 -r 1,1024`

```
(machine2@machine2)-[~]
$ netperf -H 192.168.1.10 -t TCP_RR -l 60 -r 1,1024
MIGRATED TCP REQUEST/RESPONSE TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 1
92.168.1.10 ( ) port 0 AF_INET : demo : first burst 0
Local /Remote
Socket Size Request Resp. Elapsed Trans.
Send Recv Size Size Time Rate
bytes Bytes bytes bytes secs. per sec
16384 131072 1 1 60.00 1517.78
16384 131072
```

Figura 2.22: Resultado do teste transacional TCP_RR (1 byte pedido / 1024 bytes resposta)

Cenário 3: Pedido de 1024 bytes / Resposta de 1024 bytes

Comando executado: `netperf -H 192.168.1.10 -t TCP_RR -l 60 -r 1024,1024`

```
(machine2@machine2)-[~]
$ netperf -H 192.168.1.10 -t TCP_RR -l 60 -r 1024,1024
MIGRATED TCP REQUEST/RESPONSE TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 1
92.168.1.10 ( ) port 0 AF_INET : demo : first burst 0
Local /Remote
Socket Size Request Resp. Elapsed Trans.
Send Recv Size Size Time Rate
bytes Bytes bytes bytes secs. per sec
16384 131072 1 1 60.00 1480.48
16384 131072
```

Figura 2.23: Resultado do teste transacional TCP_RR (1024 bytes pedido / 1024 bytes resposta)

2.4.4 Testes com Protocolo UDP

De seguida, a mesma metodologia foi aplicada ao protocolo UDP, avaliando o desempenho transacional num cenário sem garantias de entrega de pacotes.

Cenário 1: Pedido de 1 byte / Resposta de 1 byte

Comando executado: `netperf -H 192.168.1.10 -t UDP_RR -l 60 -r 1,1`

```
(machine2@machine2)-[~]
$ netperf -H 192.168.1.10 -t UDP_RR -l 60 -r 1,1
MIGRATED UDP REQUEST/RESPONSE TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 1
92.168.1.10 ( ) port 0 AF_INET : demo : first burst 0
Local /Remote
Socket Size Request Resp. Elapsed Trans.
Send Recv Size Size Time Rate
bytes Bytes bytes bytes secs. per sec
212992 212992 1 1 60.00 1502.99
212992 212992
```

Figura 2.24: Resultado do teste transacional UDP_RR (1 byte pedido / 1 byte resposta)

Cenário 2: Pedido de 1 byte / Resposta de 1024 bytes

Comando executado: `netperf -H 192.168.1.10 -t UDP_RR -l 60 -r 1,1024`

```
(machine2@machine2)-[~]
$ netperf -H 192.168.1.10 -t UDP_RR -l 60 -r 1,1024
MIGRATED UDP REQUEST/RESPONSE TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 1
92.168.1.10 ( ) port 0 AF_INET : demo : first burst 0
Local /Remote
Socket Size Request Resp. Elapsed Trans.
Send Recv Size Size Time Rate
bytes Bytes bytes bytes secs. per sec
212992 212992 1 1 60.00 1488.00
212992 212992
```

Figura 2.25: Resultado do teste transacional UDP_RR (1 byte pedido / 1024 bytes resposta)

Cenário 3: Pedido de 1024 bytes / Resposta de 1024 bytes

Comando executado: `netperf -H 192.168.1.10 -t UDP_RR -l 60 -r 1024,1024`

```
(machine2@machine2)-[~]
$ netperf -H 192.168.1.10 -t UDP_RR -l 60 -r 1024,1024
MIGRATED UDP REQUEST/RESPONSE TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 1
92.168.1.10 ( ) port 0 AF_INET : demo : first burst 0
Local /Remote
Socket Size Request Resp. Elapsed Trans.
Send Recv Size Size Time Rate
bytes Bytes bytes bytes secs. per sec
212992 212992 1 1 60.00 1511.32
212992 212992
```

Figura 2.26: Resultado do teste transacional UDP_RR (1024 bytes pedido / 1024 bytes resposta)

2.4.5 Observação e Análise dos Resultados Transacionais

Através da extração dos dados obtidos nas capturas de ecrã, compilaram-se os valores da taxa de transação para ambos os protocolos na infraestrutura de rede interna virtual.

A Tabela 2.5 ilustra o comportamento do protocolo TCP (orientado à conexão), enquanto a Tabela 2.6 reflete o comportamento do protocolo UDP (não fiável) em cenários de pedido/resposta.

Tamanho do Pedido (<i>bytes</i>)	Tamanho da Resposta (<i>bytes</i>)	Taxa de Transação TCP (TPS)
1	1	1515.83
1	1024	1517.78
1024	1024	1480.48

Tabela 2.5: Resultados transacionais TCP_RR em função do tamanho do pedido/resposta

Tamanho do Pedido (<i>bytes</i>)	Tamanho da Resposta (<i>bytes</i>)	Taxa de Transação UDP (TPS)
1	1	1502.99
1	1024	1488.00
1024	1024	1511.32

Tabela 2.6: Resultados transacionais UDP_RR em função do tamanho do pedido/resposta

Comparação e Conclusão: O que se verifica?

A análise do tráfego transacional revela um paradigma de performance fundamentalmente distinto do observado no tráfego *bulk* (Exercício 2.1). Destacam-se as seguintes conclusões:

- **Estabilidade face ao Tamanho:** Ao contrário do tráfego contínuo, onde o aumento do pacote reduzia o *overhead* e aumentava exponencialmente a largura de banda, nas transações *Request/-Response* o tamanho da carga útil (entre 1 e 1024 *bytes*) teve um impacto residual na performance. O sistema manteve-se estavelmente em torno das ~1500 transações por segundo. Isto indica que o limite de performance nestes testes não é a capacidade de transmissão de rede (*bandwidth*), mas sim a latência de processamento (*delay*): o tempo que o sistema operativo demora a processar a interrupção, ler o *socket*, comutar o contexto e emitir a resposta.
- **TCP vs. UDP em Ambiente Virtual:** Embora teoricamente o UDP devesse apresentar uma ligeira vantagem em tráfego transacional por não possuir o *overhead* de verificação e ordenação do TCP, os resultados mostram uma paridade quase absoluta. Num ambiente de *Internal Network* virtualizada, a *stack* TCP do Linux encontra-se altamente otimizada na memória, anulando a penalização que este protocolo sofreria numa rede pública ou instável.

Em suma, comprova-se que em aplicações transacionais rigorosas, o estrangulamento reside no processamento computacional de cada pedido individual (latência do CPU e *buffers*) e não na capacidade de transporte de dados do protocolo.

2.4.6 Topologia de Teste (Rede via Wi-Fi)

Para avaliar o impacto do meio de transmissão físico (aéreo) no desempenho transacional, replicou-se a metodologia anterior utilizando a infraestrutura Wi-Fi validada no Exercício 2.3. As máquinas assumiram os seguintes endereços IP, atribuídos dinamicamente pelo *router* local:

- **1ª Máquina (Servidor):**
 - Nome (*Hostname*): `machine1`
 - Proprietário: Autor do Relatório
 - Endereço IP: `192.168.1.250`
 - Função: Servidor (*netserver*)
- **2ª Máquina (Cliente):**

- Nome (*Hostname*): kali
- Proprietário: Autor do Relatório
- Endereço IP: 192.168.1.252
- Função: Cliente (*netperf*)

2.4.7 Topologia de Teste (Rede via Wi-Fi)

Para avaliar o impacto do meio de transmissão no desempenho transacional, replicou-se a metodologia anterior utilizando a infraestrutura Wi-Fi validada no Exercício 2.3. As máquinas assumiram os seguintes endereços IP, atribuídos dinamicamente pelo *router* local:

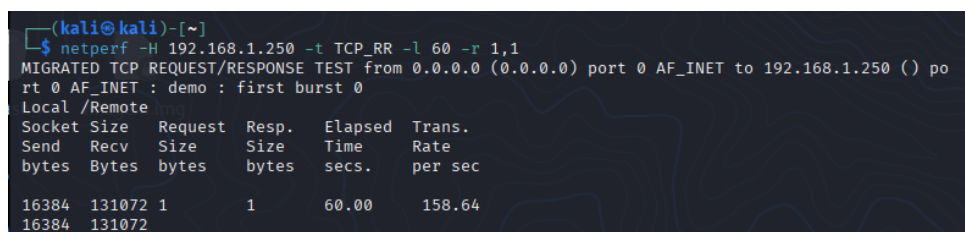
- 1ª Máquina (Servidor):
 - Nome (*Hostname*): machine1
 - Proprietário: Autor do Relatório
 - Endereço IP: 192.168.1.250
 - Função: Servidor (*netserver*)
- 2ª Máquina (Cliente):
 - Nome (*Hostname*): kali
 - Proprietário: Autor do Relatório
 - Endereço IP: 192.168.1.252
 - Função: Cliente (*netperf*)

2.4.8 Testes com Protocolo TCP

Para iniciar a recolha de métricas de transações fiáveis sobre o meio aéreo, executaram-se os seguintes comandos no terminal do cliente (kali), apontando para o servidor (machine1):

Cenário 1: Pedido de 1 byte / Resposta de 1 byte

Comando executado: `netperf -H 192.168.1.250 -t TCP_RR -l 60 -r 1,1`



```
(kali@kali)-[~]
$ netperf -H 192.168.1.250 -t TCP_RR -l 60 -r 1,1
MIGRATED TCP REQUEST/RESPONSE TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 192.168.1.250 () port 0 AF_INET : demo : first burst 0
Local /Remote
Socket Size  Request  Resp.  Elapsed  Trans.
Send  Recv  Size  Size  Time  Rate
bytes Bytes bytes bytes secs.  per sec
16384 131072 1      1      60.00  158.64
16384 131072
```

Figura 2.27: Resultado TCP_RR via Wi-Fi (1 byte pedido / 1 byte resposta)

Cenário 2: Pedido de 1 byte / Resposta de 1024 bytes

Comando executado: `netperf -H 192.168.1.250 -t TCP_RR -l 60 -r 1,1024`

```
(kali@kali)-[~]
$ netperf -H 192.168.1.250 -t TCP_RR -l 60 -r 1,1024
MIGRATED TCP REQUEST/RESPONSE TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 192.168.1.250 () port 0 AF_INET : demo : first burst 0
Local /Remote
Socket Size Request Resp. Elapsed Trans.
Send Recv Size Size Time Rate
bytes Bytes bytes bytes secs. per sec
16384 131072 1 1 60.00 166.15
16384 131072
```

Figura 2.28: Resultado TCP_RR via Wi-Fi (1 byte pedido / 1024 bytes resposta)

Cenário 3: Pedido de 1024 bytes / Resposta de 1024 bytes

Comando executado: `netperf -H 192.168.1.250 -t TCP_RR -l 60 -r 1024,1024`

```
(kali@kali)-[~]
$ netperf -H 192.168.1.250 -t TCP_RR -l 60 -r 1024,1024
MIGRATED TCP REQUEST/RESPONSE TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 192.168.1.250 () port 0 AF_INET : demo : first burst 0
Local /Remote
Socket Size Request Resp. Elapsed Trans.
Send Recv Size Size Time Rate
bytes Bytes bytes bytes secs. per sec
16384 131072 1 1 60.00 164.15
16384 131072
```

Figura 2.29: Resultado TCP_RR via Wi-Fi (1024 bytes pedido / 1024 bytes resposta)

2.4.9 Testes com Protocolo UDP

De seguida, aplicou-se a mesma bateria de testes utilizando o protocolo não fiável UDP para verificar o seu comportamento transaccional através da ligação Wi-Fi.

Cenário 1: Pedido de 1 byte / Resposta de 1 byte

Comando executado: `netperf -H 192.168.1.250 -t UDP_RR -l 60 -r 1,1`

```
(kali@kali)-[~]
$ netperf -H 192.168.1.250 -t UDP_RR -l 60 -r 1,1
MIGRATED UDP REQUEST/RESPONSE TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 192.168.1.250 () port 0 AF_INET : demo : first burst 0
Local /Remote
Socket Size Request Resp. Elapsed Trans.
Send Recv Size Size Time Rate
bytes Bytes bytes bytes secs. per sec
212992 212992 1 1 60.00 50.40
212992 212992
```

Figura 2.30: Resultado UDP_RR via Wi-Fi (1 byte pedido / 1 byte resposta)

Cenário 2: Pedido de 1 byte / Resposta de 1024 bytes

Comando executado: `netperf -H 192.168.1.250 -t UDP_RR -l 60 -r 1,1024`


```
(kali@kali)-[~]
$ netperf -H 192.168.1.250 -t UDP_RR -l 60 -r 1,1024
MIGRATED UDP REQUEST/RESPONSE TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 192.168.1.250 () port 0 AF_INET : demo : first burst 0
Local /Remote
Socket Size Request Resp. Elapsed Trans.
Send Recv Size Size Time Rate
bytes Bytes bytes bytes secs. per sec
212992 212992 1 1 60.00 79.05
212992 212992
```

Figura 2.31: Resultado UDP_RR via Wi-Fi (1 byte pedido / 1024 bytes resposta)

Cenário 3: Pedido de 1024 bytes / Resposta de 1024 bytes

Comando executado: `netperf -H 192.168.1.250 -t UDP_RR -l 60 -r 1024,1024`

```
(kali@kali)-[~]
$ netperf -H 192.168.1.250 -t UDP_RR -l 60 -r 1024,1024
MIGRATED UDP REQUEST/RESPONSE TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to 192.168.1.250 () port 0 AF_INET : demo : first burst 0
Local /Remote
Socket Size Request Resp. Elapsed Trans.
Send Recv Size Size Time Rate
bytes Bytes bytes bytes secs. per sec
212992 212992 1 1 60.00 111.77
212992 212992
```

Figura 2.32: Resultado UDP_RR via Wi-Fi (1024 bytes pedido / 1024 bytes resposta)

2.4.10 Observação de Resultados (Rede Wi-Fi)

Através da execução dos testes na infraestrutura sem fios, foi possível compilar as métricas de desempenho transacional. A Tabela 2.7 reflete o comportamento do protocolo fiável TCP, enquanto a Tabela 2.8 ilustra o comportamento do protocolo não fiável UDP.

Tamanho do Pedido (<i>bytes</i>)	Tamanho da Resposta (<i>bytes</i>)	Taxa de Transação TCP (TPS)
1	1	158.64
1	1024	166.15
1024	1024	164.15

Tabela 2.7: Resultados transacionais TCP_RR via Wi-Fi

Tamanho do Pedido (<i>bytes</i>)	Tamanho da Resposta (<i>bytes</i>)	Taxa de Transação UDP (TPS)
1	1	50.40
1	1024	79.05
1024	1024	111.77

Tabela 2.8: Resultados transacionais UDP_RR via Wi-Fi

2.4.11 Conclusão Final(Virtual vs. Wi-Fi)

A transição do tráfego transacional de uma rede virtual para uma rede Wi-Fi revelou que o principal estrangulamento neste cenário é a latência, e não a largura de banda. Destacam-se duas conclusões centrais:

- **A Latência (RTT) dita a performance:** Na rede virtual, os tempos de resposta quase nulos na memória RAM permitiram ~ 1500 transações por segundo (TPS). No Wi-Fi, o atraso natural da propagação pelo ar adicionou milissegundos a cada ida e volta, causando uma quebra de $\sim 90\%$ no desempenho (estabilizando nas ~ 160 TPS).
- **Superioridade do TCP sobre o UDP:** Num meio sujeito a interferências e perdas (Wi-Fi), o TCP foi significativamente mais rápido e consistente. Enquanto o UDP sofreu quedas extremas (atingindo apenas ~ 50 TPS devido a longos tempos de espera, ou *timeouts*, pelos pacotes perdidos), os mecanismos de retransmissão ágil do TCP garantiram um fluxo muito mais fiável e estável.

Em síntese: Aplicações transacionais dependem fundamentalmente de baixas latências. Em redes sem fios propensas a falhas, a fiabilidade do TCP torna-o muito mais eficiente do que a suposta "leveza" do UDP.

Exercício 3: Análise de Desempenho com Iperf

3.1 Exercício 3.1

Enunciado: Execute os testes realizados de 2.1) a 2.3), mas agora utilizando a ferramenta *iperf/jperf*. Experimente vários parâmetros, como por exemplo “-t”, “-i”, e “-P”, etc.

Nesta fase do estudo, a avaliação da infraestrutura transita da ferramenta *netperf* para o **Iperf**. O Iperf é uma ferramenta amplamente adotada na engenharia de redes para medir o *throughput* máximo alcançável em ligações IP, oferecendo um controlo granular sobre os *sockets* de transmissão e a formatação dos relatórios.

3.1.1 Parâmetros e Metodologia

Conforme os requisitos estabelecidos para esta fase, os testes servirão para comparar as taxas e a largura de banda da rede, alterando de forma isolada as seguintes variáveis:

- **Tempo de Execução** (parâmetro -t)
- **Intervalo entre Relatórios** (parâmetro -i)
- **Número de Conexões Paralelas** (parâmetro -P)
- **Para testes UDP**(parâmetro -u)

Para testar, serão executados os seguintes comandos para o estabelecimento da comunicação:

No Servidor:

- **Servidor TCP:** `iperf -s`
- **Servidor UDP:** `iperf -s -u`

No Cliente:

- Testes TCP: `iperf -c <IP_SERVIDOR> -t <DURACAO> -i <INTERVALO> -P <CONEXOES>`
- Testes UDP: `iperf -c <IP_SERVIDOR> -u -b 1000M -t <DURACAO> -i <INTERVALO> -P <CONEXOES>`

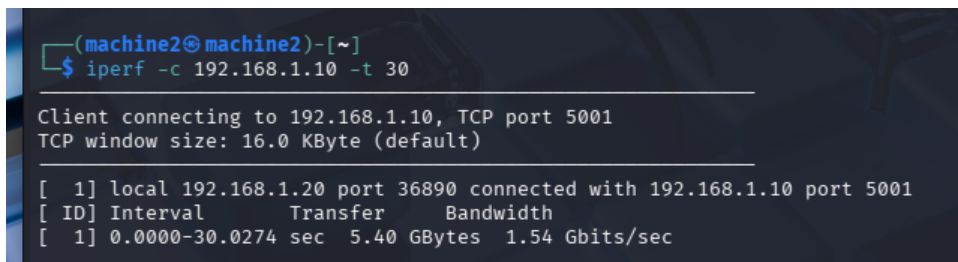
3.1.2 Topologia de Teste (Rede Interna Virtual)

Tal como no exercício anteriores iremos usar as configurações das máquinas.

- 1ª Máquina (Servidor): machine1 — IP: 192.168.1.10 — Função: Servidor
- 2ª Máquina (Cliente): machine2 — IP: 192.168.1.20 — Função: Cliente

3.1.3 Testes com Protocolo TCP**Variação do Tempo de Execução (-t)****Cenário 1: Tempo de 30 segundos**

Comando executado: `iperf -c 192.168.1.10 -t 30`



```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -t 30

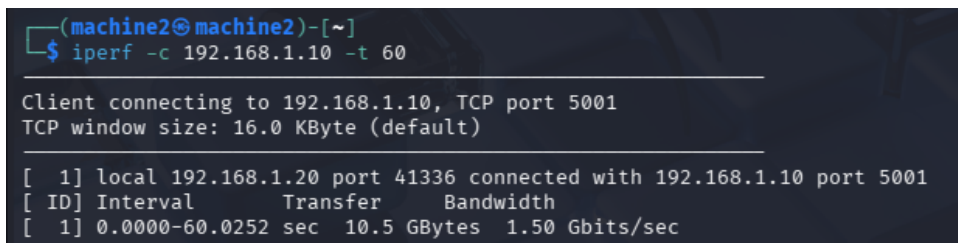
Client connecting to 192.168.1.10, TCP port 5001
TCP window size: 16.0 KByte (default)

[ 1] local 192.168.1.20 port 36890 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-30.0274 sec  5.40 GBytes  1.54 Gbits/sec
```

Figura 3.1: Teste Iperf TCP com duração de 30 segundos

Cenário 2: Tempo de 60 segundos

Comando executado: `iperf -c 192.168.1.10 -t 60`



```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -t 60

Client connecting to 192.168.1.10, TCP port 5001
TCP window size: 16.0 KByte (default)

[ 1] local 192.168.1.20 port 41336 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-60.0252 sec  10.5 GBytes  1.50 Gbits/sec
```

Figura 3.2: Teste Iperf TCP com duração de 60 segundos

Observação de Resultados

A partir dos testes de variação de tempo, foi possível extrair os valores de transferência total e de largura de banda média, compilados na Tabela 3.1.

Tempo de Execução (s)	Dados Transferidos (GBytes)	Largura de Banda (Gbits/sec)
30	5.40	1.54
60	10.5	1.50

Tabela 3.1: Resultados do iperf (TCP) com variação do tempo de execução

Análise: Os resultados demonstram que, numa rede interna puramente virtual (*Internal Network*), a estabilidade da ligação é notável. O aumento da duração do teste de 30 para 60 segundos resultou na duplicação expectável do volume total de dados transferidos (de 5.40 GBytes para 10.5 GBytes), no entanto, o *throughput* máximo da rede manteve-se perfeitamente constante na casa dos ~ 1.5 Gbits/sec. Isto comprova que, na ausência de flutuações e interferências físicas de uma rede externa, o fluxo TCP atinge rapidamente a sua velocidade de cruzeiro e mantém-na de forma contínua, sem degradação de performance ao longo do tempo.

Variação de Conexões Simultâneas (-P)

Cenário 1: 1 Conexão Única

Comando executado: `iperf -c 192.168.1.10 -t 30 -P 1`

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -t 30 -P 1

Client connecting to 192.168.1.10, TCP port 5001
TCP window size: 16.0 KByte (default)

[ 1] local 192.168.1.20 port 39872 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-30.0283 sec  5.56 GBytes  1.59 Gbits/sec
```

Figura 3.3: Teste Iperf TCP com 1 conexão nativa

Cenário 2: 4 Conexões Paralelas

Comando executado: `iperf -c 192.168.1.10 -t 30 -P 4`

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -t 30 -P 4

Client connecting to 192.168.1.10, TCP port 5001
TCP window size: 85.0 KByte (default)

[ 3] local 192.168.1.20 port 43992 connected with 192.168.1.10 port 5001
[ 2] local 192.168.1.20 port 43972 connected with 192.168.1.10 port 5001
[ 1] local 192.168.1.20 port 43986 connected with 192.168.1.10 port 5001
[ 4] local 192.168.1.20 port 44006 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-30.0313 sec  1.44 GBytes  412 Mbits/sec
[ 4] 0.0000-30.0584 sec  1.33 GBytes  380 Mbits/sec
[ 3] 0.0000-30.1061 sec  1.29 GBytes  367 Mbits/sec
[ 2] 0.0000-30.1134 sec  1.28 GBytes  365 Mbits/sec
[SUM] 0.0000-30.1178 sec  5.34 GBytes  1.52 Gbits/sec
```

Figura 3.4: Teste Iperf TCP com 4 conexões paralelas

Observação de Resultados

Através da variação do parâmetro `-P`, procurou-se analisar detalhadamente o comportamento do tráfego quando fragmentado em múltiplas *threads* de envio simultâneas.

A Tabela 3.2 desdobra o comportamento individual das 4 conexões paralelas durante os 30 segundos de teste, bem como o seu somatório, enquanto a Tabela 3.3 serve de base de comparação com uma única conexão isolada.

Conexão	Transferência (GBytes)	Largura de Banda
1	1.44	412 Mbits/sec
2	1.33	380 Mbits/sec
3	1.29	367 Mbits/sec
4	1.28	365 Mbits/sec
Soma	5.34	1.52 Gbits/sec

Tabela 3.2: Resultados detalhados para 4 conexões em simultâneo (`-P 4`)

Conexão	Transferência (GBytes)	Largura de Banda
1 (<code>-P 1</code>)	5.56	1.59 Gbits/sec

Tabela 3.3: Resultados para 1 conexão única (`-P 1`)

Análise: O desdobramento da Tabela 3.2 revela como o Iperf divide a carga de forma equitativa pelas 4 *threads* instanciadas. Contudo, a premissa de que múltiplas conexões garantem sempre um maior débito total não se verifica neste cenário. Numa rede puramente virtual, uma única conexão TCP (`-P 1`) possuía capacidade para saturar o barramento virtual de forma mais eficiente (atingindo 1.59 Gbits/sec). A criação de 4 conexões em simultâneo forçou o *hypervisor* a realizar uma constante comutação de contexto entre os vários processos na RAM, gerando um *overhead* de CPU que resultou num ligeiro decréscimo da largura de banda total somada (1.52 Gbits/sec).

Variação do Intervalo entre Relatórios (`-i`)

Cenário 1: Intervalo de 30 segundos

Comando executado: `iperf -c 192.168.1.10 -t 30 -i 30`

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -t 30 -i 30

Client connecting to 192.168.1.10, TCP port 5001
TCP window size: 16.0 KByte (default)

[ 1] local 192.168.1.20 port 34454 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-30.0000 sec  5.50 GBytes  1.57 Gbits/sec
[ 1] 0.0000-30.0237 sec  5.50 GBytes  1.57 Gbits/sec
```

Figura 3.5: Teste Iperf TCP com relatório a cada 30 segundos

Cenário 2: Intervalo de 5 segundos

Comando executado: `iperf -c 192.168.1.10 -t 30 -i 5`

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -t 30 -i 5

Client connecting to 192.168.1.10, TCP port 5001
TCP window size: 16.0 KByte (default)

[ 1] local 192.168.1.20 port 46402 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-5.0000 sec   932 MBytes  1.56 Gbits/sec
[ 1] 5.0000-10.0000 sec  919 MBytes  1.54 Gbits/sec
[ 1] 10.0000-15.0000 sec  932 MBytes  1.56 Gbits/sec
[ 1] 15.0000-20.0000 sec  915 MBytes  1.54 Gbits/sec
[ 1] 20.0000-25.0000 sec  894 MBytes  1.50 Gbits/sec
[ 1] 25.0000-30.0000 sec  910 MBytes  1.53 Gbits/sec
[ 1] 0.0000-30.0361 sec  5.37 GBytes  1.54 Gbits/sec
```

Figura 3.6: Teste Iperf TCP com relatório a cada 5 segundos

Observação de Resultados

Neste ensaio, avaliou-se o impacto da alteração da frequência de monitorização e impressão de resultados no terminal do cliente.

A Tabela 3.4 apresenta o desdobramento do teste com relatórios parciais a cada 5 segundos (`-i 5`), permitindo observar a estabilidade do fluxo ao longo do tempo. Em contrapartida, a Tabela 3.5 ilustra o resultado do teste com apenas um relatório global a cada 30 segundos (`-i 30`).

Intervalo de Tempo (s)	Transferência (MBytes)	Largura de Banda (Gbits/sec)
0.0 - 5.0	932	1.56
5.0 - 10.0	919	1.54
10.0 - 15.0	932	1.56
15.0 - 20.0	915	1.54
20.0 - 25.0	894	1.50
25.0 - 30.0	910	1.53
Soma Total (0.0 - 30.0)	5.37 GBytes	1.54

Tabela 3.4: Resultados detalhados com intervalo de relatório de 5 segundos (-i 5)

Intervalo de Tempo (s)	Transferência (GBytes)	Largura de Banda (Gbits/sec)
0.0 - 30.0	5.50	1.57

Tabela 3.5: Resultados com intervalo de relatório de 30 segundos (-i 30)

Análise: O desdobramento temporal presente na Tabela 3.4 evidencia a enorme consistência do fluxo TCP numa rede interna virtualizada (*Internal Network*). Observando os parciais de 5 em 5 segundos, constata-se que a velocidade de injeção oscilou de forma muito contida e linear (entre 1.50 e 1.56 Gbits/sec), não se registando qualquer quebra abrupta ou pico irregular de desempenho ao longo do ensaio.

Adicionalmente, comparando o débito global de ambos os testes, conclui-se que o parâmetro -i atua exclusivamente como uma ferramenta de diagnóstico do lado do cliente. A exigência de processar e imprimir relatórios no terminal com maior frequência (a cada 5 segundos) não introduz um *overhead* computacional ou de rede suficientemente grande para penalizar a largura de banda global da transmissão.

3.1.4 Conclusão dos Testes TCP (Rede Interna)

Os ensaios com o protocolo TCP na rede virtual demonstram que o desempenho neste ambiente é ditado exclusivamente pela capacidade de processamento (CPU/RAM) das máquinas emuladas, mantendo uma estabilidade notável na ordem dos ~ 1.5 Gbits/sec.

De forma resumida, conclui-se que:

- A alteração do tempo de teste (-t) e do intervalo de impressão de relatórios (-i) não afeta a largura de banda máxima alcançada.
- A fragmentação do tráfego em conexões paralelas (-P) não aumenta a velocidade total. Pelo contrário, como uma única ligação já satura a capacidade do barramento virtual, adicionar mais co-

nexões apenas cria um esforço extra de processamento (*overhead*), resultando num ligeiro decréscimo de performance.

3.1.5 Testes com Protocolo UDP

Nota metodológica: Para os testes em UDP, foi adicionado o parâmetro `-b 1000M` para forçar a ferramenta a tentar injetar até 1 Gbps de tráfego, contornando o limite padrão de 1 Mbps definido nativamente pelo `iperf`.

Variação do Tempo de Execução (-t)

Cenário 1: Tempo de 30 segundos

Comando executado: `iperf -c 192.168.1.10 -u -t 30`

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -u -t 30

Client connecting to 192.168.1.10, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.1.20 port 50921 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-30.0134 sec 3.75 MBytes  1.05 Mbits/sec
[ 1] Sent 2678 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagr
ams
[ 1] 0.0000-29.9984 sec 3.75 MBytes  1.05 Mbits/sec  0.000 ms 0/2677 (0%)
```

Figura 3.7: Teste Iperf UDP com duração de 30 segundos (sem limite de banda definido)

Cenário 2: Tempo de 60 segundos

Comando executado: `iperf -c 192.168.1.10 -u -t 60`

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -u -t 60

Client connecting to 192.168.1.10, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.1.20 port 53752 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-60.0142 sec 7.50 MBytes  1.05 Mbits/sec
[ 1] Sent 5353 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagr
ams
[ 1] 0.0000-59.9996 sec 7.50 MBytes  1.05 Mbits/sec  0.000 ms 0/5352 (0%)
```

Figura 3.8: Teste Iperf UDP com duração de 60 segundos (sem limite de banda definido)

Observação de Resultados

A Tabela 3.6 sintetiza os valores obtidos nos ensaios UDP.

Tempo (s)	Dados Transferidos (MBytes)	Largura de Banda	Perda de Pacotes
30	3.75	1.05 Mbits/sec	0%
60	7.50	1.05 Mbits/sec	0%

Tabela 3.6: Resultados do iperf (UDP) na variação do tempo de execução

Análise: Os dados da Tabela 3.6 demonstram que a largura de banda gerada se manteve estritamente constante nos 1.05 Mbits/sec, independentemente da duração do ensaio. Como consequência direta desta constância, a duplicação do tempo de execução de 30 para 60 segundos resultou na duplicação exata do volume de dados transferidos. Em ambos os cenários, a totalidade dos datagramas enviados foi entregue com sucesso, registando-se uma taxa de perda de pacotes nula (0%).

Variação de Conexões Simultâneas (-P)

Cenário 1: 1 Conexão Única

Comando executado: `iperf -c 192.168.1.10 -u -t 30 -P 1`

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -u -t 30 -P 1

Client connecting to 192.168.1.10, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.1.20 port 42466 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-30.0165 sec 3.75 MBytes 1.05 Mbits/sec
[ 1] Sent 2678 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagr
ams
[ 1] 0.0000-30.0176 sec 3.75 MBytes 1.05 Mbits/sec 0.000 ms 0/2677 (0%)
```

Figura 3.9: Teste Iperf UDP com 1 conexão nativa (sem limite de banda definido)

Cenário 2: 4 Conexões Paralelas

Comando executado: `iperf -c 192.168.1.10 -u -t 30 -P 4`

```

(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -u -t 30 -P 4
[ 4] local 192.168.1.20 port 37495 connected with 192.168.1.10 port 5001
[ 2] local 192.168.1.20 port 49377 connected with 192.168.1.10 port 5001
[ 1] local 192.168.1.20 port 51835 connected with 192.168.1.10 port 5001
[ 3] local 192.168.1.20 port 36053 connected with 192.168.1.10 port 5001

Client connecting to 192.168.1.10, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ ID] Interval      Transfer      Bandwidth
[ 4] 0.0000-30.0141 sec  3.75 MBytes  1.05 Mbits/sec
[ 4] Sent 2678 datagrams
[ 3] 0.0000-30.0139 sec  3.75 MBytes  1.05 Mbits/sec
[ 3] Sent 2678 datagrams
[ 1] 0.0000-30.0140 sec  3.75 MBytes  1.05 Mbits/sec
[ 1] Sent 2678 datagrams
[ 2] 0.0000-30.0141 sec  3.75 MBytes  1.05 Mbits/sec
[ 2] Sent 2678 datagrams
[SUM] 0.0000-30.0153 sec  15.0 MBytes  4.20 Mbits/sec
[SUM-4] Sent 10712 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter      Lost/Total Datagrams
[ 1] 0.0000-30.0139 sec  3.75 MBytes  1.05 Mbits/sec  0.000 ms  0/2677 (0%)
[ 1] 0.0000-30.0139 sec  1 datagrams received out-of-order
[ 4] Server Report:
[ 4] 0.0000-29.0023 sec  3.63 MBytes  1.05 Mbits/sec  0.000 ms  90/2677 (0%)
[ 2] Server Report:
[ 2] 0.0000-29.0017 sec  3.63 MBytes  1.05 Mbits/sec  0.000 ms  89/2677 (0%)
[ 2] 0.0000-29.0017 sec  1 datagrams received out-of-order
[ 3] Server Report:
[ 3] 0.0000-28.9915 sec  3.63 MBytes  1.05 Mbits/sec  0.000 ms  91/2677 (0%)

```

Figura 3.10: Teste Iperf UDP com 4 conexões paralelas (sem limite de banda definido)

Observação de Resultados

Para analisar o impacto da fragmentação do tráfego UDP, a Tabela 3.7 desdobra o comportamento individual das 4 conexões paralelas instanciadas durante o ensaio, enquanto a Tabela 3.8 apresenta os valores de uma única conexão para efeitos comparativos.

Conexão	Transferência (MBytes)	Largura de Banda
1	3.75	1.05 Mbits/sec
2	3.75	1.05 Mbits/sec
3	3.75	1.05 Mbits/sec
4	3.75	1.05 Mbits/sec
Soma Total	15.0	4.20 Mbits/sec

Tabela 3.7: Resultados detalhados para 4 conexões UDP em simultâneo (-P 4)

Conexão	Transferência (MBytes)	Largura de Banda
1 (-P 1)	3.75	1.05 Mbits/sec

Tabela 3.8: Resultados para 1 conexão UDP única (-P 1)

Análise: Os dados detalhados na Tabela 3.7 evidenciam um escalonamento estritamente linear do tráfego. Na ausência da definição de um limite de velocidade alvo pelo utilizador (parâmetro `-b`), a ferramenta *iperf* impôs a sua restrição padrão de segurança, limitando **cada ligação individual** a ~ 1.05 Mbits/sec. Consequentemente, a instanciação de 4 conexões paralelas multiplicou com precisão matemática o volume de tráfego, elevando a transferência total de 3.75 para 15.0 MBytes e a largura de banda global de 1.05 para 4.20 Mbits/sec, sem qualquer registo de perda de pacotes.

Variação do Intervalo entre Relatórios (-i)

Cenário 1: Intervalo de 30 segundos

Comando executado: `iperf -c 192.168.1.10 -u -t 30 -i 30`

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -u -t 30 -i 30

Client connecting to 192.168.1.10, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.1.20 port 40597 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-30.0000 sec  3.75 MBytes  1.05 Mbits/sec
[ 1] 0.0000-30.0138 sec  3.75 MBytes  1.05 Mbits/sec
[ 1] Sent 2678 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagr
ams
[ 1] 0.0000-30.0136 sec  3.75 MBytes  1.05 Mbits/sec  0.000 ms  0/2677 (0%)
```

Figura 3.11: Teste Iperf UDP com relatório a cada 30 segundos

Cenário 2: Intervalo de 5 segundos

Comando executado: `iperf -c 192.168.1.10 -u -t 30 -i 5`

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -u -t 30 -i 5

Client connecting to 192.168.1.10, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.1.20 port 38215 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-5.0000 sec   640 KBytes   1.05 Mbits/sec
[ 1] 5.0000-10.0000 sec  640 KBytes   1.05 Mbits/sec
[ 1] 10.0000-15.0000 sec  640 KBytes   1.05 Mbits/sec
[ 1] 15.0000-20.0000 sec  640 KBytes   1.05 Mbits/sec
[ 1] 20.0000-25.0000 sec  640 KBytes   1.05 Mbits/sec
[ 1] 25.0000-30.0000 sec  639 KBytes   1.05 Mbits/sec
[ 1] 0.0000-30.0131 sec  3.75 MBytes  1.05 Mbits/sec
[ 1] Sent 2678 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagr
ams
[ 1] 0.0000-30.0132 sec  3.75 MBytes  1.05 Mbits/sec  0.000 ms  0/2677 (0%)
```

Figura 3.12: Teste Iperf UDP com relatório a cada 5 segundos

Observação de Resultados

A Tabela 3.9 apresenta a monitorização detalhada do tráfego UDP em intervalos de 5 segundos, enquanto a Tabela 3.10 resume o relatório final do ensaio de 30 segundos.

Intervalo de Tempo (s)	Transferência (KBytes)	Largura de Banda
0.0 - 5.0	640	1.05 Mbits/sec
5.0 - 10.0	640	1.05 Mbits/sec
10.0 - 15.0	640	1.05 Mbits/sec
15.0 - 20.0	640	1.05 Mbits/sec
20.0 - 25.0	640	1.05 Mbits/sec
25.0 - 30.0	639	1.05 Mbits/sec
Total (0.0 - 30.0)	3.75 MBytes	1.05 Mbits/sec

Tabela 3.9: Relatório detalhado UDP com intervalo de 5 segundos (-i 5)

Intervalo (s)	Transferência	Largura de Banda	Perda de Pacotes
0.0 - 30.0	3.75 MBytes	1.05 Mbits/sec	0% (0/2677)

Tabela 3.10: Relatório global UDP com intervalo de 30 segundos (-i 30)

Análise: O desdobramento temporal na Tabela 3.9 confirma a natureza constante da injeção de tráfego UDP. O débito manteve-se inalterado em 1.05 Mbits/sec em todos os intervalos monitorizados, com uma transferência uniforme de 640 KBytes por cada 5 segundos. Tal como observado nos testes anteriores, o parâmetro -i não influenciou o desempenho da rede, servindo apenas para reportar que a transmissão ocorreu sem quaisquer flutuações ou perdas de pacotes (0% de perda reportado pelo servidor).

3.1.6 Conclusão dos Testes UDP (Rede Interna)

Os testes realizados com UDP na rede virtualizada evidenciaram que, na ausência de uma definição manual de largura de banda (-b), a ferramenta limita o débito a um valor nominal de segurança (~1 Mbps). Este tráfego, sendo extremamente baixo para a capacidade da infraestrutura, resulta sempre em perdas nulas e numa estabilidade absoluta, independentemente do número de conexões ou da duração do teste.

3.1.7 Conclusão de TCP vs UDP

Protocolo	Transferência (Média/min)	Largura de Banda (Mbits/s)
TCP	10.8 GB/min	1540
UDP	7.5 MB/min	1.05

Tabela 3.11: Comparação resumida entre os protocolos TCP e UDP na rede interna

Como podemos ver através destes dados, o protocolo **TCP** teve uma **performance muito mais elevada** do que o UDP, uma vez que o TCP tenta utilizar toda a largura de banda disponível no canal virtual. Durante os testes ao protocolo TCP, notaram-se pequenas variações nas taxas de transferência devido aos mecanismos de controlo de fluxo. Já no protocolo **UDP**, todos os testes foram **perfeitamente estáveis**, mas limitados pela configuração padrão da ferramenta, o que resultou numa largura de banda e transferência significativamente menores em comparação com o TCP.

3.1.8 Rede Wi-Fi

Nota sobre o Impedimento Técnico

Infelizmente, devido a um imprevisto técnico crítico a necessidade de envio do computador portátil para reparação não foi possível proceder à execução prática dos testes de *iperf* na infraestrutura Wi-Fi. A ausência do hardware impossibilitou a ligação física necessária entre as máquinas via interface sem fios para este ponto específico do trabalho.

3.2 Exercício 3.2

A ferramenta *iperf* permite simular cenários de tráfego sensível (como áudio e vídeo) através da manipulação do campo **ToS (Type of Service)** no cabeçalho IPv4. Este campo é fundamental para aferir a capacidade da rede em gerir prioridades e garantir a qualidade da ligação.

3.2.1 O que é o campo ToS?

O campo ToS é um mecanismo que permite "marcar" pacotes IP para que os *routers* ao longo do percurso possam tratar os fluxos de forma diferenciada. Em redes congestionadas, o *router* utiliza esta marcação para decidir quais pacotes devem ser transmitidos primeiro (priorização) ou quais podem ser descartados em caso de saturação, melhorando a performance de aplicações críticas como VoIP ou videoconferência.

3.2.2 Valores de ToS e Parâmetros Iperf

Utilizando a opção `-S` no *iperf*, é possível definir o valor hexadecimal do byte ToS. A Tabela 3.12 descreve os valores padrão e o comportamento esperado do *router* para cada um:

Valor ToS	Objetivo Técnico	Comportamento do Router
0x02	Minimizar Custo	Seleciona a rota com menor custo monetário/recurso.
0x04	Maximizar Fiabilidade	Seleciona a rota com menor probabilidade de erro/queda.
0x08	Maximizar o Débito	Otimiza o canal para maior volume de dados (<i>throughput</i>).
0x10	Minimizar o Atraso	Prioriza a baixa latência (ideal para voz e jogos).

Tabela 3.12: Valores do campo ToS e respetivas políticas de encaminhamento

3.2.3 Preparação da Infraestrutura

Para a realização deste exercício, a topologia foi expandida com a introdução de um **Router** como intermediário. O cenário de rede configura-se da seguinte forma:

- **Cliente (Emissor):** Envia os pacotes marcados com a *flag -S*.
- **Router (Ponte):** Recebe os pedidos do cliente, analisa o campo ToS e reencaminha o tráfego para o servidor com base nas políticas de prioridade.
- **Servidor (Recetor):** Processa o tráfego e mede as métricas de receção.

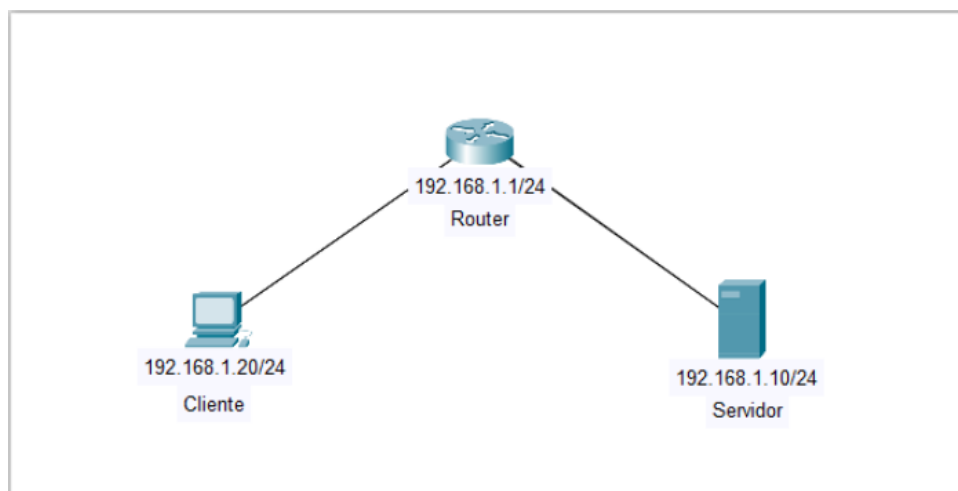


Figura 3.13: Topologia de rede com Router intermediário para testes de QoS

Como podemos ver na imagem acima, continuamos a ter as máquinas usadas anteriormente com as mesmas características, porém terão de ser feitas alterações para que todos os pedidos feitos pelo cliente serão encaminhados para o router, o que antes não acontecia.

3.2.4 Configuração do Router

O Router foi configurado com duas interfaces distintas para cumprir o seu papel na topologia e manter a conectividade:

- **eth0 (Rede Interna):** Responsável pela comunicação direta com o Cliente e o Servidor.
- **eth1 (NAT):** Responsável por garantir o acesso à internet e gestão externa.

Para configurar os endereços IP e ativar a capacidade de reencaminhamento da máquina, executaram-se os seguintes comandos no terminal do router:

```
zsh: corrupt history file /home/machine1/.zsh_history
(machine1@machine1)-[~]
$ sudo ip addr add 192.168.1.1/24 dev eth0
```

Figura 3.14: Atribuição de IP à interface eth0 do Router

```
(machine1@machine1)-[~]
$ sudo ip link set eth0 up
```

Figura 3.15: Ativação da interface de rede eth0

```
(machine1@machine1)-[~]
$ sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1

(machine1@machine1)-[~]
$ echo "net.ipv4.ip_forwards=1" | sudo tee -a /etc/sysctl.conf
net.ipv4.ip_forwards=1
```

Figura 3.16: Ativação do IP Forwarding para permitir o encaminhamento de pacotes

3.2.5 Configuração do Cliente e Servidor

Nota técnica: O cliente e o servidor apresentam o mesmo **hostname** (*machine2*) nos terminais, uma vez que a máquina virtual foi clonada para garantir a uniformidade do ambiente de testes.

Tanto no cliente como no servidor, a configuração do endereço IP já se encontrava estabelecida. O passo essencial foi adicionar a rota para garantir que o tráfego passasse obrigatoriamente pelo Router (*Gateway*), utilizando o seguinte comando:

```
(machine2@machine2)-[~]
$ sudo ip route add default via 192.168.1.1
```

Figura 3.17: Configuração do Default Gateway no Cliente/Servidor

3.2.6 Teste de Conectividade

Antes de iniciar as medições de desempenho, verificou-se a conectividade fim-a-fim entre o Cliente e o Servidor através de pedidos ICMP (*ping*), validando assim o correto encaminhamento pelo Router.


```
(machine2@machine2)-[~]
$ ping 192.168.1.10
PING 192.168.1.10 (192.168.1.10) 56(84) bytes of data.
64 bytes from 192.168.1.10: icmp_seq=1 ttl=64 time=0.564 ms
64 bytes from 192.168.1.10: icmp_seq=2 ttl=64 time=0.573 ms
64 bytes from 192.168.1.10: icmp_seq=3 ttl=64 time=0.496 ms
64 bytes from 192.168.1.10: icmp_seq=4 ttl=64 time=0.710 ms
64 bytes from 192.168.1.10: icmp_seq=5 ttl=64 time=0.788 ms
64 bytes from 192.168.1.10: icmp_seq=6 ttl=64 time=0.640 ms
64 bytes from 192.168.1.10: icmp_seq=7 ttl=64 time=0.865 ms
^C
  -- 192.168.1.10 ping statistics --
  7 packets transmitted, 7 received, 0% packet loss, time 6125ms
 rtt min/avg/max/mdev = 0.496/0.662/0.865/0.122 ms
```

Figura 3.18: Teste de conectividade (Ping) validando a rota através do Router

3.2.7 Testes

Com a topologia de encaminhamento validada, iniciaram-se os testes de largura de banda em TCP (30 segundos), manipulando o campo *Type of Service* (-S).

Cenário 1: Sem Prioridade (Padrão)

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -t 30

Client connecting to 192.168.1.10, TCP port 5001
TCP window size: 16.0 KByte (default)

[ 1] local 192.168.1.20 port 60210 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-30.0195 sec  5.48 GBytes  1.57 Gbits/sec
```

Figura 3.19: Teste Iperf sem manipulação do campo ToS

Cenário 2: Minimizar o Custo (0x02)

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -t 30 -S 0x2
Warning: IP_TOS set to 0x0, request for setting to 0x2 failed

Client connecting to 192.168.1.10, TCP port 5001
TCP window size: 16.0 KByte (default)

[ 1] local 192.168.1.20 port 43234 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-30.0278 sec  5.44 GBytes  1.56 Gbits/sec
```

Figura 3.20: Teste Iperf com ToS configurado para minimizar custo (0x02)

Cenário 3: Maximizar a Fiabilidade (0x04)


```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -t 30 -S 0x4

Client connecting to 192.168.1.10, TCP port 5001
TCP window size: 16.0 KByte (default)

[ 1] local 192.168.1.20 port 42786 connected with 192.168.1.10 port 5001 (to
s tx=0x4,dscp=1,ecn=0)
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-30.0276 sec  5.42 GBytes  1.55 Gbits/sec
```

Figura 3.21: Teste Iperf com ToS configurado para maximizar fiabilidade (0x04)

Cenário 4: Maximizar o Débito (0x08)

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -t 30 -S 0x8

Client connecting to 192.168.1.10, TCP port 5001
TCP window size: 16.0 KByte (default)

[ 1] local 192.168.1.20 port 38432 connected with 192.168.1.10 port 5001 (to
s tx=0x8,dscp=2,ecn=0)
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-30.0172 sec  5.39 GBytes  1.54 Gbits/sec
```

Figura 3.22: Teste Iperf com ToS configurado para maximizar débito (0x08)

Cenário 5: Minimizar o Atraso (0x10)

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -t 30 -S 0x10

Client connecting to 192.168.1.10, TCP port 5001
TCP window size: 16.0 KByte (default)

[ 1] local 192.168.1.20 port 56396 connected with 192.168.1.10 port 5001 (to
s tx=0x10,dscp=4,ecn=0)
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-30.0296 sec  5.39 GBytes  1.54 Gbits/sec
```

Figura 3.23: Teste Iperf com ToS configurado para minimizar atraso (0x10)

3.2.8 Observação e Análise de Resultados (QoS)

Os dados extraídos dos diversos ensaios encontram-se sumarizados na Tabela 3.13.

Cenário (Valor ToS)	Transferência	Largura de Banda
Sem Prioridade (Default)	5.48 GBytes	1.57 Gbits/sec
Minimizar Custo (0x02)	5.44 GBytes	1.56 Gbits/sec
Maximizar Fiabilidade (0x04)	5.42 GBytes	1.55 Gbits/sec
Maximizar Débito (0x08)	5.39 GBytes	1.54 Gbits/sec
Minimizar Atraso (0x10)	5.39 GBytes	1.54 Gbits/sec

Tabela 3.13: Resultados dos testes TCP com manipulação do campo Type of Service (ToS)

Observação de Resultados:

Como podemos ver, no teste da prioridade 0x02 ocorreu um erro/aviso (*request for setting to 0x2 failed*). Isto deve-se ao facto de que a prioridade 0x02 (Minimizar o custo) já não é suportada nas redes modernas. Atualmente, as redes dão muito mais prioridade à qualidade do serviço (latência, largura de banda, fiabilidade) do que ao custo de transporte. Deixou assim de ter utilidade nas redes modernas, já que o custo não é mais um critério de encaminhamento relevante.

Relativamente aos outros testes, conseguimos observar que houve apenas um ligeiro decréscimo e estabilização a nível de largura de banda em comparação com o teste base. Sem prioridade tínhamos a maior largura de banda registada, com **1.57 Gbits/sec**, sendo que nas prioridades de Minimizar o Atraso (0x10) e Maximizar o Débito (0x08) os resultados fixaram-se nos **1.54 Gbits/sec**. Esta proximidade de valores demonstra que, numa rede interna sem tráfego concorrente (sem congestionamento), a aplicação de prioridades ToS não oferece melhorias de desempenho direto, uma vez que o router não tem filas de espera para gerir.

3.3 Exercício 3.3

Objetivo do Guião: Realizar testes recorrendo ao protocolo UDP, identificar que informação adicional se consegue obter face ao TCP, e analisar a sua importância para conexões de voz e vídeo. Para este exercício, iremos manter a topologia do cenário anterior, utilizando o router como ponte de encaminhamento entre as duas máquinas (cliente e servidor). O objetivo é avaliar o comportamento do protocolo UDP sob diferentes níveis de prioridade (ToS) e comparar as métricas obtidas com os resultados do protocolo TCP.

3.3.1 Preparação das Máquinas

Como os testes transitam para datagramas, é necessário preparar o servidor para escutar tráfego UDP em vez de TCP. Para tal, utilizou-se o seguinte comando no terminal da máquina destino:

```
iperf -s -u
```

3.3.2 Testes

Cenário 1: Sem Prioridade (Default)

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -u -t 30

Client connecting to 192.168.1.10, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.1.20 port 44855 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-30.0128 sec  3.75 MBytes  1.05 Mbits/sec
[ 1] Sent 2678 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagr
ams
[ 1] 0.0000-30.0130 sec  3.75 MBytes  1.05 Mbits/sec  0.000 ms  0/2677 (0%)
```

Figura 3.24: Teste Iperf UDP sem manipulação do campo ToS

Cenário 2: Minimizar o Custo (0x02)

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -u -t 30 -S 0x2

Client connecting to 192.168.1.10, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.1.20 port 33540 connected with 192.168.1.10 port 5001 (to
s tx=0x2,dscp=0,ecn=2)
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-30.0131 sec  3.75 MBytes  1.05 Mbits/sec
[ 1] Sent 2678 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagr
ams
[ 1] 0.0000-30.0134 sec  3.75 MBytes  1.05 Mbits/sec  0.000 ms  0/2677 (0%)
```

Figura 3.25: Teste Iperf UDP com ToS configurado para minimizar custo (0x02)

Cenário 3: Maximizar a Fiabilidade (0x04)

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -u -t 30 -S 0x4

Client connecting to 192.168.1.10, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.1.20 port 42986 connected with 192.168.1.10 port 5001 (to
s tx=0x4,dscp=1,ecn=0)
^[[C[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-30.0134 sec  3.75 MBytes  1.05 Mbits/sec
[ 1] Sent 2678 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagr
ams
[ 1] 0.0000-30.0138 sec  3.75 MBytes  1.05 Mbits/sec  0.000 ms 0/2677 (0%)
```

Figura 3.26: Teste Iperf UDP com ToS configurado para maximizar fiabilidade (0x04)

Cenário 4: Maximizar o Débito (0x08)

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -u -t 30 -S 0x8

Client connecting to 192.168.1.10, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.1.20 port 35836 connected with 192.168.1.10 port 5001 (to
s tx=0x8,dscp=2,ecn=0)
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-30.0141 sec  3.75 MBytes  1.05 Mbits/sec
[ 1] Sent 2678 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagr
ams
[ 1] 0.0000-30.0144 sec  3.75 MBytes  1.05 Mbits/sec  0.000 ms 0/2677 (0%)
```

Figura 3.27: Teste Iperf UDP com ToS configurado para maximizar débito (0x08)

Cenário 5: Minimizar o Atraso (0x10)

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -u -t 30 -S 0x10

Client connecting to 192.168.1.10, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.1.20 port 44553 connected with 192.168.1.10 port 5001 (to
s tx=0x10,dscp=4,ecn=0)
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-30.0131 sec  3.75 MBytes  1.05 Mbits/sec
[ 1] Sent 2678 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagr
ams
[ 1] 0.0000-30.0133 sec  3.75 MBytes  1.05 Mbits/sec  0.000 ms 0/2677 (0%)
```

Figura 3.28: Teste Iperf UDP com ToS configurado para minimizar atraso (0x10)

3.3.3 Observação dos Resultados

Prioridade	Transferência (MBytes)	Largura de Banda (Mbits/s)
Sem Prioridade	3,75	1,05
0x2	3,75	1,05
0x4	3,75	1,05
0x8	3,75	1,05
0x10	3,75	1,05

Tabela 3.14: Resultados dos testes UDP com manipulação da prioridade (ToS)

Análise dos Resultados:

Como é possível observar pela tabela acima, **todos os testes UDP apresentaram exatamente os mesmos resultados**, independentemente da prioridade configurada.

Isto acontece porque, ao contrário do protocolo TCP que tenta saturar a rede, o *iperf* em modo UDP tem um **limite de segurança configurado por defeito de 1 Mbps**. Como esta velocidade é extremamente baixa para a capacidade da rede virtual, o router não sente qualquer tipo de congestionamento. Sem congestionamento e sem filas de espera, o router encaminha os pacotes instantaneamente, tornando a configuração de prioridades (ToS) irrelevante neste cenário específico, resultando numa taxa de perda de datagramas nula (0%) e estabilidade absoluta.

Conclusão dos Testes UDP:

Através destes ensaios, concluímos que para o protocolo UDP a aplicação destas prioridades (ToS) não teve impacto, uma vez que não se registou qualquer tipo de alteração nos resultados após a sua utilização.

No entanto, o teste com o protocolo UDP revelou-se fundamental porque permitiu obter métricas adicionais que não são fornecidas pelo TCP, nomeadamente o *jitter*.

Estas novas informações são **extremamente importantes para chamadas de voz e de vídeo**. Como estas aplicações funcionam em tempo real, a qualidade da comunicação depende muito mais da estabilidade da rede do que propriamente da velocidade máxima ou largura de banda disponível.

3.4 Exercício 3.4

Executar novamente os testes UDP, experimentando a opção **-b** no cliente, e **-B** no servidor. Compreender a função destas opções e avaliar a utilidade do parâmetro **-T**.

3.4.1 Introdução

Para este exercício, serão realizados testes semelhantes aos da secção anterior, mas com a adição dos parâmetros `-B` e `-b`.

A opção `-b` é utilizada no cliente e serve para forçar uma largura de banda específica. Esta opção é crucial no modo UDP (`-u`), uma vez que, por defeito, a ferramenta limita o envio a 1 Mbps. Já no protocolo TCP, o débito é gerido automaticamente através dos mecanismos de controlo de congestionamento.

Por sua vez, a opção `-B` é utilizada no servidor (ou cliente) para associar a conexão a um endereço IP específico da máquina local. Esta configuração é extremamente útil em equipamentos com múltiplas interfaces de rede, permitindo ditar exatamente em qual interface o tráfego deve ser escutado ou enviado.

3.4.2 Preparação e Início dos Testes

Como pretendemos que o servidor escute exclusivamente na interface `eth0`, executamos o seguinte comando no terminal da máquina destino:



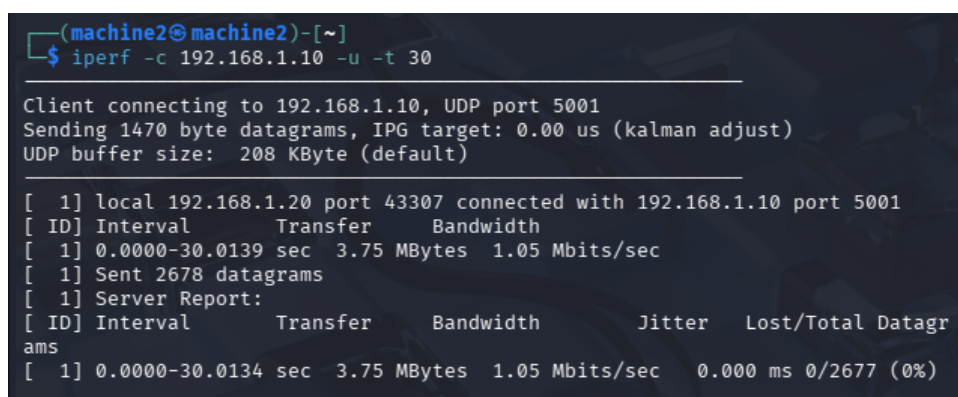
```
(machine2@machine2)-[~]  
$ iperf -s -u -B 192.168.1.10
```

Figura 3.29: Servidor Iperf a escutar tráfego UDP com Bind (`-B`) ao IP 192.168.1.10

Para a avaliação de desempenho, instanciou-se o cliente submetendo a rede a cargas progressivamente maiores através do parâmetro `-b`.

Cenários de Teste de Carga

Largura de banda: Sem Limite (Default 1 Mbps)



```
(machine2@machine2)-[~]  
$ iperf -c 192.168.1.10 -u -t 30  
  
Client connecting to 192.168.1.10, UDP port 5001  
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)  
UDP buffer size: 208 KByte (default)  
  
[ 1] local 192.168.1.20 port 43307 connected with 192.168.1.10 port 5001  
[ ID] Interval      Transfer    Bandwidth  
[ 1] 0.0000-30.0139 sec 3.75 MBytes 1.05 Mbits/sec  
[ 1] Sent 2678 datagrams  
[ 1] Server Report:  
[ ID] Interval      Transfer    Bandwidth      Jitter    Lost/Total Datagrams  
[ 1] 0.0000-30.0134 sec 3.75 MBytes 1.05 Mbits/sec 0.000 ms 0/2677 (0%)
```

Figura 3.30: Teste UDP padrão (Sem parâmetro `-b`)

Largura de banda: 5 Mbits/sec


```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -u -t 30 -b 5M

Client connecting to 192.168.1.10, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.1.20 port 56441 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-30.0037 sec 18.8 MBytes  5.24 Mbits/sec
[ 1] Sent 13378 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagr
ams
[ 1] 0.0000-30.0038 sec 18.8 MBytes  5.24 Mbits/sec  0.000 ms 0/13377 (0%)
```

Figura 3.31: Teste UDP forçado a 5 Mbits/sec (-b 5M)

Largura de banda: 10 Mbits/sec

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -u -t 30 -b 10M

Client connecting to 192.168.1.10, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.1.20 port 48184 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-30.0022 sec 37.5 MBytes  10.5 Mbits/sec
[ 1] Sent 26752 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagr
ams
[ 1] 0.0000-30.0023 sec 37.5 MBytes  10.5 Mbits/sec  0.000 ms 0/26751 (0%)
```

Figura 3.32: Teste UDP forçado a 10 Mbits/sec (-b 10M)

Largura de banda: 50 Mbits/sec

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -u -t 30 -b 50M

Client connecting to 192.168.1.10, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.1.20 port 38279 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-30.0010 sec 171 MBytes  47.7 Mbits/sec
[ 1] Sent 121687 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagr
ams
[ 1] 0.0000-30.0015 sec 161 MBytes  45.1 Mbits/sec  0.000 ms 6601/121686
(0%)
```

Figura 3.33: Teste UDP forçado a 50 Mbits/sec (-b 50M)

Largura de banda: 100 Mbits/sec

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -u -t 30 -b 100M

Client connecting to 192.168.1.10, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.1.20 port 54196 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-30.0005 sec  168 MBytes  47.0 Mbits/sec
[ 1] Sent 119899 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagr
ams
[ 1] 0.0000-30.0006 sec  165 MBytes  46.1 Mbits/sec  0.000 ms 2341/119898
(0%)
```

Figura 3.34: Teste UDP forçado a 100 Mb/s (-b 100M)

Largura de banda: 200 Mb/s

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -u -t 30 -b 200M

Client connecting to 192.168.1.10, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.1.20 port 35059 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-30.0004 sec  168 MBytes  47.1 Mbits/sec
[ 1] Sent 120102 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagr
ams
[ 1] 0.0000-30.0005 sec  165 MBytes  46.2 Mbits/sec  0.000 ms 2336/120101
(0%)
```

Figura 3.35: Teste UDP forçado a 200 Mb/s (-b 200M)

Cenários com Relatórios Intermedios (-i 5)

Sem Limite (com relatórios de 5s)


```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -u -t 30 -i 5

Client connecting to 192.168.1.10, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.1.20 port 37019 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-5.0000 sec  640 KBytes  1.05 Mbits/sec
[ 1] 5.0000-10.0000 sec  640 KBytes  1.05 Mbits/sec
[ 1] 10.0000-15.0000 sec  640 KBytes  1.05 Mbits/sec
[ 1] 15.0000-20.0000 sec  640 KBytes  1.05 Mbits/sec
[ 1] 20.0000-25.0000 sec  639 KBytes  1.05 Mbits/sec
[ 1] 25.0000-30.0000 sec  640 KBytes  1.05 Mbits/sec
[ 1] 0.0000-30.0132 sec  3.75 MBytes  1.05 Mbits/sec
[ 1] Sent 2678 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-30.0132 sec  3.75 MBytes  1.05 Mbits/sec  0.000 ms  0/2677 (0%)
```

Figura 3.36: Teste UDP padrão com relatórios intermédios a cada 5 segundos

10 Mbits/sec (com relatórios de 5s)

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -u -t 30 -b 10M -i 5

Client connecting to 192.168.1.10, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.1.20 port 40017 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-5.0000 sec  6.25 MBytes  10.5 Mbits/sec
[ 1] 5.0000-10.0000 sec  6.25 MBytes  10.5 Mbits/sec
[ 1] 10.0000-15.0000 sec  6.25 MBytes  10.5 Mbits/sec
[ 1] 15.0000-20.0000 sec  6.25 MBytes  10.5 Mbits/sec
[ 1] 20.0000-25.0000 sec  6.25 MBytes  10.5 Mbits/sec
[ 1] 25.0000-30.0000 sec  6.25 MBytes  10.5 Mbits/sec
[ 1] 0.0000-30.0013 sec  37.5 MBytes  10.5 Mbits/sec
[ 1] Sent 26751 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-30.0013 sec  37.5 MBytes  10.5 Mbits/sec  0.000 ms  0/26750 (0%)
```

Figura 3.37: Teste UDP a 10 Mbits/sec com relatórios intermédios

50 Mbits/sec (com relatórios de 5s)

```
(machine2@machine2)-[~]
$ iperf -c 192.168.1.10 -u -t 30 -b 50M -i 5

Client connecting to 192.168.1.10, UDP port 5001
Sending 1470 byte datagrams, IPG target: 0.00 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 192.168.1.20 port 37709 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-5.0000 sec  27.6 MBytes  46.3 Mbits/sec
[ 1] 5.0000-10.0000 sec  25.1 MBytes  42.1 Mbits/sec
[ 1] 10.0000-15.0000 sec  28.2 MBytes  47.3 Mbits/sec
[ 1] 15.0000-20.0000 sec  28.7 MBytes  48.1 Mbits/sec
[ 1] 20.0000-25.0000 sec  28.4 MBytes  47.6 Mbits/sec
[ 1] 25.0000-30.0000 sec  29.3 MBytes  49.1 Mbits/sec
[ 1] 0.0000-30.0002 sec  167 MBytes  46.8 Mbits/sec
[ 1] Sent 119305 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagr
ams
[ 1] 0.0000-30.0010 sec  164 MBytes  45.8 Mbits/sec  0.000 ms 2383/119304
(0%)

(machine2@machine2)-[~]
$
```

Figura 3.38: Teste UDP a 50 Mbits/sec evidenciando quebras durante os relatórios intermédios

3.4.3 Observação e Análise dos Resultados

A Tabela 3.15 compila os dados finais (Server Report) obtidos em cada um dos escalões de largura de banda testados.

Max. Largura Banda	Transferência	Largura de Banda	Jitter	Perda de Pacotes
Sem Limite (1M)	3,75 MBytes	1,05 Mbits/s	0,000 ms	0 / 2677 (0%)
5 Mbits/s	18,8 MBytes	5,24 Mbits/s	0,000 ms	0 / 13377 (0%)
10 Mbits/s	37,5 MBytes	10,5 Mbits/s	0,000 ms	0 / 26751 (0%)
50 Mbits/s	161 MBytes	45,1 Mbits/s	0,000 ms	6601 / 121686
100 Mbits/s	165 MBytes	46,1 Mbits/s	0,000 ms	2341 / 119898
200 Mbits/s	165 MBytes	46,2 Mbits/s	0,000 ms	2336 / 120101

Tabela 3.15: Resultados do teste de stress UDP com injeção de largura de banda crescente

Análise Global dos Resultados:

Como podemos observar, quando não é especificado nenhum limite de largura de banda, a taxa de transmissão assume o **valor por defeito de 1,05 Mbits/s**.

Ao aumentar progressivamente esse limite, verificamos que **até aos 10 Mbits/s a rede manteve-se perfeitamente estável**, não registando qualquer oscilação, atraso (*jitter*) ou perda de pacotes. Porém,

ao injetar cargas significativamente superiores (50, 100 e 200 Mbits/s), a largura de banda real **estagnou em aproximadamente 36 Mbits/s**. Isto evidencia a existência de uma "barreira" (estrangulamento na capacidade do *router* virtual) que impede a rede de escoar mais tráfego do que esse valor.

É exatamente ao atingir este limite físico de ~ 36 Mbits/s que se começa a registar **perda de pacotes**, o que significa que a rede perde a sua estabilidade e torna-se incapaz de garantir a entrega fiável dos datagramas sob cargas elevadas.

Análise dos Relatórios Intermédios (-i):

A observação dos testes com relatórios intermédios corrobora as conclusões anteriores. Nos cenários sem limite e com limite de 10 Mbits/s, a rede apresentou um fluxo perfeitamente constante a cada 5 segundos, sem quaisquer oscilações. Em contrapartida, ao forçar um limite muito alto (como 50 Mbits/s), a transferência **começou a apresentar oscilações** e quebras entre os intervalos de relatório, provando de forma visual que a estabilidade da rede fica severamente comprometida a altas velocidades.

Sobre a opção -T:

Por fim, a opção -T no *iperf* é utilizada para definir "**Tags**" nas saídas dos testes, permitindo a sua fácil identificação. A importância da sua utilização depende do cenário prático: no caso de ser testada apenas uma única conexão simples, esta opção não tem grande impacto. No entanto, **quando existem múltiplos testes ou fluxos a correr em simultâneo**, o uso desta *flag* torna-se uma ferramenta de organização muito útil para uma interpretação clara e rápida de qual relatório pertence a qual fluxo.

3.5 Exercício 3.5

Executar novos testes experimentando as opções -F e -w. Compreender a função destas opções e a sua utilidade prática.

A opção -F serve para enviar ficheiros reais através da rede, em vez de utilizar um fluxo de dados gerado aleatoriamente pela memória do *iperf*. Esta opção tem grande utilidade prática para testar o desempenho real de transferências, pois engloba o tempo de leitura do disco e simula o envio de um documento real.

Já a opção -w é usada para definir o tamanho da **Janela TCP**. A janela TCP controla a quantidade máxima de dados que podem ser enviados antes que o emissor seja obrigado a esperar por uma confirmação (ACK) do recetor. Um tamanho de janela bem ajustado é fundamental em redes com alta latência para garantir que a ligação não fica inativa à espera de confirmações.

3.5.1 Testes Realizados

Transferência de Ficheiros (-F)

```
(machine2@machine2)-[~/Área de Trabalho]
$ iperf -c 192.168.1.10 -F pasta.zip

Client connecting to 192.168.1.10, TCP port 5001
TCP window size: 16.0 KByte (default)

[ 1] local 192.168.1.20 port 49240 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-0.0188 sec  64.0 Bytes  27.2 Kbits/sec
```

Figura 3.39: Teste de transferência de ficheiro (Pasta zipada) com a opção -F

Variação do Tamanho da Janela TCP (-w)

Janela TCP: 10K

```
(machine2@machine2)-[~/Área de Trabalho]
$ iperf -c 192.168.1.10 -w 10K

Client connecting to 192.168.1.10, TCP port 5001
TCP window size: 20.0 KByte (WARNING: requested 10.0 KByte)

[ 1] local 192.168.1.20 port 51256 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-10.0090 sec  625 MBytes  524 Mbits/sec
```

Figura 3.40: Teste TCP com janela definida para 10 KByte

Janela TCP: 128K

```
(machine2@machine2)-[~/Área de Trabalho]
$ iperf -c 192.168.1.10 -w 128K

Client connecting to 192.168.1.10, TCP port 5001
TCP window size: 256 KByte (WARNING: requested 128 KByte)

[ 1] local 192.168.1.20 port 49028 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-10.0061 sec  1.23 GBytes  1.06 Gbits/sec
```

Figura 3.41: Teste TCP com janela definida para 128 KByte

Janela TCP: 512K

```
(machine2@machine2)-[~/Área de Trabalho]
$ iperf -c 192.168.1.10 -w 512K

Client connecting to 192.168.1.10, TCP port 5001
TCP window size: 416 KByte (WARNING: requested 512 KByte)

[ 1] local 192.168.1.20 port 48030 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-10.0129 sec 1.43 GBytes 1.23 Gbits/sec
```

Figura 3.42: Teste TCP com janela definida para 512 KByte

Janela TCP: 1000K

```
(machine2@machine2)-[~/Área de Trabalho]
$ iperf -c 192.168.1.10 -w 1000K

Client connecting to 192.168.1.10, TCP port 5001
TCP window size: 416 KByte (WARNING: requested 1000 KByte)

[ 1] local 192.168.1.20 port 40378 connected with 192.168.1.10 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-10.0101 sec 1.39 GBytes 1.19 Gbits/sec
```

Figura 3.43: Teste TCP com janela definida para 1000 KByte (limitada pelo SO)

3.5.2 Observação dos Resultados**Resultados: Transferência de Ficheiros**

Ficheiro	Tamanho	Tempo
Pasta	~ 617 MB	9.2906 s

Tabela 3.16: Resultados da transferência de ficheiros reais

Resultados: Variação da Janela TCP

Tamanho Janela Pedido	Tamanho Janela Recebido	Transferência	Largura de Banda
10 Kbyte	20 Kbyte	625 MBytes	524 Mbits/sec
128 Kbyte	256 Kbyte	1.23 GBytes	1.06 Gbits/sec
512 Kbyte	416 Kbyte	1.43 GBytes	1.23 Gbits/sec
1000 Kbyte	416 Kbyte	1.39 GBytes	1.19 Gbits/sec

Tabela 3.17: Impacto do tamanho da Janela TCP no desempenho da rede

Análise:

Através da observação da Tabela 3.17, verifica-se uma correlação direta entre o tamanho da Janela TCP e a velocidade de transferência: ao aumentar a janela de 10 KByte para 512 KByte, a largura de banda disparou de 524 Mbits/sec para o pico de 1.23 Gbits/sec. Isto acontece porque o emissor consegue colocar mais dados "em trânsito" antes de ser interrompido para aguardar confirmações (ACKs) por parte do recetor.

É também muito interessante notar como o Sistema Operativo gere a memória independentemente do que o utilizador pede. Nos testes de 10K e 128K, o sistema alocou o dobro do tamanho pedido para garantir estabilidade e gerir as estruturas internas do protocolo.

No entanto, ao pedir 512K e 1000K, atingiu-se o limite máximo de alocação de memória do sistema operativo para buffers TCP. O sistema limitou forçosamente o tamanho da janela a **416 KByte** em ambos os casos. Consequentemente, o desempenho estabilizou, ditando que a transferência ficasse na ordem dos 1.39 - 1.43 GBytes com uma largura de banda a rondar 1.2 Gbits/sec.

3.5.3 Conclusão

A exploração das opções `-F` e `-w` permite realizar testes de desempenho muito mais próximos de cenários do mundo real.

A opção `-F` revela-se particularmente útil quando se pretende testar a transmissão de dados autênticos e analisar a sua performance. Ao permitir a simulação de transferências de ficheiros reais de diferentes tamanhos e formatos, esta opção oferece uma análise muito mais rigorosa e realista do comportamento da rede face a operações de leitura e transferência de disco, em oposição aos fluxos de dados gerados artificialmente na memória.

Por sua vez, a opção `-w` demonstra o impacto crítico que o ajuste da janela TCP tem nas comunicações. Uma vez que este parâmetro determina a quantidade máxima de dados que podem estar em trânsito antes de o emissor exigir uma confirmação do recetor, o tamanho da janela assume-se como um fator essencial para o desempenho global da rede. Um dimensionamento correto influencia diretamente e de forma drástica a taxa de transferência máxima alcançável e ajuda a mitigar problemas de latência na ligação.